

ProcessHub

v. 1.1.0

Nguyen Huynh Tri Cuong

06.02.2026

Contents

1	Introduction	1
1.1	Introduction	1
1.1.1	Target Audience	1
1.1.2	Prerequisites	1
2	Description	2
2.1	Description	2
2.1.1	Introduction	2
2.1.2	Architecture Overview	2
2.1.3	Component Architecture	3
2.1.4	Communication Patterns	5
2.1.5	Restart Coordination Protocol	7
2.1.6	Protocol Messages	7
2.1.7	Web Dashboard	8
2.1.8	Logging Integration	9
2.1.9	Fleet Orchestrator	9
2.1.10	Installation	12
2.1.11	Quick Start	12
2.1.12	Testing	13
2.1.13	Extensibility	14
2.1.14	Design Improvements	15
2.1.15	Diagrams Reference	15
3	__init__.py	18
4	__init__.py	19
5	loader.py	20
5.1	Function: expand_vars	20
5.2	Function: fully_expand_vars	20
5.3	Function: expand_and_quote	20
5.4	Function: compute_restart_sequence	21
5.5	Class: ConfigLoader	21
5.6	Class: JsonConfigLoader	21
5.6.1	Method: load	22
5.7	Class: JsonPreprocessorLoader	22
5.7.1	Method: load	22
5.8	Class: ProcessConfigLoader	22

5.8.1	Method: <code>config</code>	22
5.8.2	Method: <code>load_file</code>	22
5.8.3	Method: <code>load_dict</code>	23
5.8.4	Method: <code>get_process_names</code>	23
5.8.5	Method: <code>get_process_config</code>	23
5.8.6	Method: <code>has_process</code>	23
5.8.7	Method: <code>get_process_arguments</code>	23
5.8.8	Method: <code>get_executor_config</code>	24
6	<code>schemas.py</code>	25
6.1	Function: <code>validate_process_config</code>	25
6.2	Function: <code>validate_server_config</code>	25
6.3	Function: <code>validate_client_config</code>	26
6.4	Class: <code>ProcessConfig</code>	26
6.4.1	Method: <code>to_executor_config</code>	26
6.5	Class: <code>ProcessesConfig</code>	26
6.5.1	Method: <code>get</code>	27
6.5.2	Method: <code>names</code>	27
6.5.3	Method: <code>from_dict</code>	27
6.6	Class: <code>TransportConfig</code>	27
6.7	Class: <code>LoggingServerConfig</code>	27
6.8	Class: <code>ServerConfig</code>	27
6.8.1	Method: <code>from_dict</code>	28
6.9	Class: <code>ClientConfig</code>	28
6.9.1	Method: <code>from_dict</code>	28
7	<code>__init__.py</code>	29
8	<code>hub_core.py</code>	30
8.1	Class: <code>MessageType</code>	30
8.2	Class: <code>OutgoingMessage</code>	30
8.3	Class: <code>ProcessExecutorProtocol</code>	30
8.3.1	Method: <code>start_process</code>	31
8.3.2	Method: <code>stop_process</code>	31
8.3.3	Method: <code>is_running</code>	31
8.4	Class: <code>ConnectionInfo</code>	31
8.5	Class: <code>ProcessHubCore</code>	32
8.5.1	Method: <code>get_state_snapshot</code>	32
8.5.2	Method: <code>handle_register</code>	33
8.5.3	Method: <code>handle_unregister</code>	33
8.5.4	Method: <code>handle_connection_info</code>	33
8.5.5	Method: <code>is_registered</code>	33
8.5.6	Method: <code>handle_start_request</code>	34
8.5.7	Method: <code>handle_stop_request</code>	34
8.5.8	Method: <code>handle_restart_ready</code>	34
8.5.9	Method: <code>tick</code>	34
8.5.10	Method: <code>get_connection_count</code>	35

8.5.11	Method: <code>get_process_count</code>	35
8.5.12	Method: <code>register_admin_process</code>	35
8.5.13	Method: <code>unregister_admin_process</code>	35
8.5.14	Method: <code>reset</code>	36
8.5.15	Method: <code>shutdown</code>	36
9	<code>models.py</code>	37
9.1	Class: <code>ProcessState</code>	37
9.2	Class: <code>ProcessInfo</code>	37
9.2.1	Method: <code>add_requester</code>	38
9.2.2	Method: <code>remove_requester</code>	38
9.2.3	Method: <code>is_active</code>	38
9.2.4	Method: <code>to_dict</code>	38
9.3	Class: <code>MessageBase</code>	38
9.4	Class: <code>RegisterConnectionRequest</code>	39
9.5	Class: <code>RegisterConnectionResponse</code>	39
9.6	Class: <code>UnregisterConnectionRequest</code>	39
9.7	Class: <code>UnregisterConnectionResponse</code>	39
9.8	Class: <code>ConnectionInfoRequest</code>	39
9.9	Class: <code>ConnectionInfoResponse</code>	39
9.10	Class: <code>ProcessStartRequest</code>	39
9.11	Class: <code>ProcessStartResponse</code>	40
9.12	Class: <code>ProcessStopRequest</code>	40
9.13	Class: <code>ProcessStopResponse</code>	40
9.14	Class: <code>ProcessRestartNotify</code>	40
9.15	Class: <code>ProcessRestartReady</code>	41
9.16	Class: <code>ProcessRestartDone</code>	41
9.17	Class: <code>ServerShutdownNotify</code>	41
9.18	Class: <code>ConnectionSnapshot</code>	41
9.19	Class: <code>ProcessSnapshot</code>	41
9.20	Class: <code>HubStateSnapshot</code>	41
10	<code>process_registry.py</code>	43
10.1	Class: <code>ProcessRegistry</code>	43
10.1.1	Method: <code>get</code>	44
10.1.2	Method: <code>exists</code>	44
10.1.3	Method: <code>get_all_names</code>	44
10.1.4	Method: <code>get_snapshot</code>	44
10.1.5	Method: <code>get_requesters</code>	45
10.1.6	Method: <code>get_state</code>	45
10.1.7	Method: <code>register</code>	45
10.1.8	Method: <code>unregister</code>	45
10.1.9	Method: <code>remove</code>	46
10.1.10	Method: <code>update_state</code>	46
10.1.11	Method: <code>update_pid</code>	47
10.1.12	Method: <code>remove_requester_from_all</code>	47
10.1.13	Method: <code>get_active_processes</code>	47

10.1.14 Method: <code>mark_dead</code>	47
10.1.15 Method: <code>clear_all</code>	47
11 <code>restart_state.py</code>	48
11.1 Class: <code>RestartState</code>	48
11.2 Class: <code>RestartContext</code>	48
11.2.1 Method: <code>reset</code>	49
11.2.2 Method: <code>is_timed_out</code>	49
11.2.3 Method: <code>can_retry</code>	49
11.2.4 Method: <code>elapsed_time</code>	49
11.3 Class: <code>RestartStateMachine</code>	49
11.3.1 Method: <code>state</code>	50
11.3.2 Method: <code>killed_processes</code>	50
11.3.3 Method: <code>pending_panels</code>	50
11.3.4 Method: <code>get_snapshot</code>	50
11.3.5 Method: <code>detect_killed</code>	50
11.3.6 Method: <code>notifications_sent</code>	51
11.3.7 Method: <code>panel_acknowledged</code>	51
11.3.8 Method: <code>begin_restart</code>	51
11.3.9 Method: <code>restart_completed</code>	51
11.3.10 Method: <code>reset_to_idle</code>	52
11.3.11 Method: <code>check_timeout</code>	52
11.3.12 Method: <code>force_fail</code>	52
12 <code>__init__.py</code>	53
13 <code>fleet_client.py</code>	54
13.1 Class: <code>FleetClient</code>	54
13.1.1 Method: <code>get_fleet_status</code>	54
13.1.2 Method: <code>get_hub_status</code>	54
13.1.3 Method: <code>start_processes</code>	55
13.1.4 Method: <code>stop_processes</code>	55
13.1.5 Method: <code>reset_hub</code>	56
13.1.6 Method: <code>get_hub_detail</code>	56
13.1.7 Method: <code>start_on_all_hubs</code>	56
13.1.8 Method: <code>stop_all_processes</code>	56
13.1.9 Method: <code>reset_all_hubs</code>	57
13.1.10 Method: <code>on_hub_online</code>	57
13.1.11 Method: <code>on_hub_offline</code>	57
14 <code>hub_agent.py</code>	58
14.1 Class: <code>HubAgent</code>	58
14.1.1 Method: <code>hub_id</code>	58
14.1.2 Method: <code>stop</code>	58
15 <code>hub_registry.py</code>	59
15.1 Class: <code>_HubEntry</code>	59
15.1.1 Method: <code>register</code>	59

15.1.2	Method: deregister	60
15.1.3	Method: update_status	60
15.1.4	Method: update_heartbeat	60
15.1.5	Method: get_hub	60
15.1.6	Method: get_all_hubs	61
15.1.7	Method: get_online_hubs	61
15.1.8	Method: is_registered	61
15.1.9	Method: check_health	61
15.1.10	Method: get_snapshot	61
15.1.11	Method: hub_count	62
16	models.py	63
16.1	Class: HubAnnounce	63
16.2	Class: HubRegistered	63
16.3	Class: HubDeregistered	63
16.4	Class: HubStatusReport	63
16.5	Class: FleetCommand	64
16.6	Class: FleetCommandResult	64
16.7	Class: FleetStatusRequest	64
16.8	Class: FleetStatusResponse	64
16.9	Class: HubSnapshot	64
16.10	Class: FleetStateSnapshot	65
17	orchestrator.py	66
17.1	Class: FleetOrchestrator	66
17.1.1	Method: registry	66
17.1.2	Method: stop	66
17.1.3	Method: tick	67
17.1.4	Method: send_command	67
17.1.5	Method: get_fleet_snapshot	67
17.1.6	Method: get_hub_snapshot	67
18	topics.py	68
18.1	Class: FleetTopics	68
19	web_api.py	69
19.1	Class: CommandRequest	69
19.1.1	Method: start	69
20	__init__.py	70
21	integration.py	71
21.1	Class: LoggingServerType	71
21.2	Class: LoggingConfig	71
21.2.1	Method: to_command_args	71
21.2.2	Method: to_dict	71
21.3	Class: LoggingServerProtocol	72
21.3.1	Method: hot_cfg_dir	72

21.3.2	Method: <code>hot_lock_cfg_path</code>	72
21.3.3	Method: <code>allocate</code>	72
21.3.4	Method: <code>deallocate</code>	72
21.4	Class: <code>LoggingServerBase</code>	72
21.4.1	Method: <code>hot_cfg_dir</code>	72
21.4.2	Method: <code>hot_lock_cfg_path</code>	72
21.4.3	Method: <code>allocate</code>	72
21.4.4	Method: <code>deallocate</code>	72
21.5	Class: <code>MockLoggingServer</code>	73
21.5.1	Method: <code>allocate</code>	73
21.5.2	Method: <code>deallocate</code>	73
21.6	Class: <code>LoggingIntegration</code>	73
21.6.1	Method: <code>server</code>	74
21.6.2	Method: <code>config_dir</code>	74
21.6.3	Method: <code>is_started</code>	74
21.6.4	Method: <code>start</code>	74
21.6.5	Method: <code>stop</code>	74
21.6.6	Method: <code>get_process_logging_config</code>	74
21.6.7	Method: <code>enhance_process_config</code>	74
21.7	Class: <code>LoggingAwareExecutor</code>	74
21.7.1	Method: <code>start</code>	75
21.7.2	Method: <code>stop</code>	75
21.7.3	Method: <code>is_running</code>	75
22	<code>__init__.py</code>	76
23	<code>executor.py</code>	77
23.1	Function: <code>create_executor</code>	77
23.2	Class: <code>ProcessExecutor</code>	77
23.2.1	Method: <code>start</code>	77
23.2.2	Method: <code>stop</code>	78
23.2.3	Method: <code>is_running</code>	78
23.2.4	Method: <code>get_pid</code>	78
23.3	Class: <code>SimpleExecutor</code>	78
23.3.1	Method: <code>start</code>	78
23.3.2	Method: <code>stop</code>	79
23.3.3	Method: <code>is_running</code>	79
23.3.4	Method: <code>get_pid</code>	79
23.3.5	Method: <code>update_process</code>	79
23.4	Class: <code>MockExecutor</code>	79
23.4.1	Method: <code>start</code>	79
23.4.2	Method: <code>stop</code>	79
23.4.3	Method: <code>is_running</code>	79
23.4.4	Method: <code>get_pid</code>	79
23.4.5	Method: <code>kill</code>	79
24	<code>manager.py</code>	80

24.1 Class: ProcessManager	80
24.1.1 Method: executor	81
24.1.2 Method: registry	81
24.1.3 Method: register	81
24.1.4 Method: unregister	81
24.1.5 Method: start	82
24.1.6 Method: stop	82
24.1.7 Method: force_stop_all	82
24.1.8 Method: is_running	82
24.1.9 Method: get_state	82
24.1.10 Method: get_info	82
24.1.11 Method: get_all_names	82
24.1.12 Method: get_requesters	82
24.1.13 Method: health_check	83
24.1.14 Method: remove_requester_from_all	83
24.1.15 Method: get_shared_info	83
24.1.16 Method: update_process	83
25 __init__.py	84
25.1 Function: get_platform_executor	84
26 base.py	85
26.1 Class: PlatformExecutor	85
26.1.1 Method: start	85
26.1.2 Method: stop	86
26.1.3 Method: is_running	86
26.1.4 Method: get_pid	86
26.1.5 Method: update_process	86
26.1.6 Method: cleanup	86
26.1.7 Method: cleanup_all	86
27 unix.py	87
27.1 Class: UnixExecutor	87
27.1.1 Method: start	87
27.1.2 Method: stop	88
27.1.3 Method: is_running	88
27.1.4 Method: find_process	88
28 windows.py	89
28.1 Class: WindowsExecutor	89
28.1.1 Method: start	89
28.1.2 Method: stop	89
28.1.3 Method: is_running	90
28.1.4 Method: get_pid	90
28.1.5 Method: update_process	90
29 __init__.py	91

30 client.py	92
30.1 Class: ProcessHubClient	92
30.1.1 Method: start	93
30.1.2 Method: stop	93
30.1.3 Method: panel_id	93
30.1.4 Method: is_registered	93
30.1.5 Method: is_running	93
30.1.6 Method: register_connection	93
30.1.7 Method: wait_for_registration	93
30.1.8 Method: unregister_connection	93
30.1.9 Method: request_connection_info	93
30.1.10 Method: request_process_start	94
30.1.11 Method: request_process_stop	94
30.1.12 Method: notify_restart_ready	94
31 client_controller.py	95
31.1 Class: ClientEvent	95
31.2 Class: ClientController	95
31.2.1 Method: connect	96
31.2.2 Method: disconnect	96
31.2.3 Method: on_connection_registered	96
31.2.4 Method: on_connection_unregistered	96
31.2.5 Method: on_connection_info	97
31.2.6 Method: on_process_started	97
31.2.7 Method: on_process_stopped	97
31.2.8 Method: on_process_killed	97
31.2.9 Method: on_process_restarted	98
31.2.10 Method: on_restart_failed	98
31.2.11 Method: on_server_timeout	98
31.2.12 Method: on_server_shutdown	98
32 server.py	99
32.1 Class: Topics	99
32.2 Class: ProcessExecutor	99
32.2.1 Method: start	99
32.2.2 Method: stop	99
32.2.3 Method: is_running	100
32.3 Class: ProcessHubServer	100
32.3.1 Method: start	100
32.3.2 Method: stop	100
32.3.3 Method: run	100
32.3.4 Method: reset	101
32.3.5 Method: register_admin_process	101
32.3.6 Method: unregister_admin_process	101
32.3.7 Method: core	101
32.3.8 Method: configured_process_names	101
32.3.9 Method: process_config	102

32.3.10 Method: <code>is_running</code>	102
33 <code>__init__.py</code>	103
34 <code>base.py</code>	104
34.1 Class: <code>TransportBase</code>	104
34.1.1 Method: <code>start</code>	105
34.1.2 Method: <code>stop</code>	105
34.1.3 Method: <code>register_handler</code>	105
34.1.4 Method: <code>send</code>	105
34.1.5 Method: <code>send_async</code>	105
35 <code>eventbus_transport.py</code>	106
35.1 Class: <code>EventBusConfig</code>	106
35.1.1 Method: <code>to_dict</code>	107
35.2 Class: <code>ProcessHubMessage</code>	107
35.2.1 Method: <code>from_data</code>	108
35.2.2 Method: <code>get_value</code>	108
35.3 Class: <code>EventBusTransport</code>	108
35.3.1 Method: <code>register_handler</code>	108
35.3.2 Method: <code>start</code>	109
35.3.3 Method: <code>stop</code>	109
35.3.4 Method: <code>send</code>	109
35.3.5 Method: <code>send_async</code>	109
35.3.6 Method: <code>is_started</code>	109
35.3.7 Method: <code>is_connected</code>	110
36 <code>inmemory_transport.py</code>	111
36.1 Class: <code>InMemoryTransport</code>	111
36.1.1 Method: <code>start</code>	111
36.1.2 Method: <code>stop</code>	111
36.1.3 Method: <code>register_handler</code>	111
36.1.4 Method: <code>send</code>	111
36.1.5 Method: <code>send_async</code>	112
36.1.6 Method: <code>get_sent_messages</code>	112
36.1.7 Method: <code>clear</code>	112
36.1.8 Method: <code>clear_messages</code>	112
36.1.9 Method: <code>message_count</code>	112
37 <code>zmq_transport.py</code>	113
37.1 Class: <code>TcpKeepaliveConfig</code>	113
37.1.1 Method: <code>detection_time</code>	114
37.2 Class: <code>ZmqBroker</code>	114
37.2.1 Method: <code>xpub_endpoint</code>	114
37.2.2 Method: <code>xsub_endpoint</code>	114
37.2.3 Method: <code>start</code>	114
37.2.4 Method: <code>stop</code>	114
37.3 Class: <code>ZmqAsyncClient</code>	114

37.3.1	Method: <code>register_topic</code>	115
37.3.2	Method: <code>start</code>	115
37.3.3	Method: <code>send_message</code>	115
37.3.4	Method: <code>send_message_async</code>	115
37.3.5	Method: <code>stop</code>	115
37.4	Class: <code>ZmqTransport</code>	115
37.4.1	Method: <code>register_handler</code>	116
37.4.2	Method: <code>start</code>	116
37.4.3	Method: <code>stop</code>	116
37.4.4	Method: <code>send</code>	116
37.4.5	Method: <code>send_async</code>	116
37.4.6	Method: <code>is_started</code>	116
38	<code>__init__.py</code>	117
39	<code>base.py</code>	118
39.1	Class: <code>HubView</code>	118
39.1.1	Method: <code>on_state_changed</code>	118
39.1.2	Method: <code>render</code>	118
39.2	Class: <code>HubViewBase</code>	118
39.2.1	Method: <code>render</code>	119
39.2.2	Method: <code>on_state_changed</code>	119
40	<code>console_view.py</code>	120
40.1	Class: <code>ConsoleView</code>	120
40.1.1	Method: <code>clear</code>	120
40.1.2	Method: <code>render</code>	120
41	<code>null_view.py</code>	121
41.1	Class: <code>NullView</code>	121
41.1.1	Method: <code>render</code>	121
42	<code>web_view.py</code>	122
42.1	Class: <code>WebViewLogHandler</code>	122
42.1.1	Method: <code>emit</code>	122
42.2	Class: <code>WebView</code>	122
42.2.1	Method: <code>start</code>	123
42.2.2	Method: <code>stop</code>	123
42.2.3	Method: <code>render</code>	123
42.2.4	Method: <code>url</code>	123
42.2.5	Method: <code>docs_url</code>	123
43	Glossary	124
43.1	Glossary	124
44	Appendix	125
44.1	Appendix	125
44.1.1	License	125
44.1.2	References	125

44.1.3 Repository	125
44.1.4 Contact	125
45 History	126
45.1 History	126
45.1.1 Version 1.1.0 - 06.02.2026	126
45.1.2 Version 1.0.0 - 15.01.2026	127

Chapter 1

Introduction

1.1 Introduction

ProcessHub is a Python package for process lifecycle management with ZMQ-based coordination. It provides:

- **Process Lifecycle Management:** Start, stop, and restart processes across distributed systems
- **Health Monitoring:** Automatic detection and recovery from crashed processes
- **Restart Coordination:** Multi-client restart orchestration with acknowledgment protocol
- **Pluggable Transport:** ZMQ for production, in-memory for testing
- **Decoupled Architecture:** Core logic separated from I/O for testability

1.1.1 Target Audience

This package is designed for developers who need to:

- Manage multiple processes in a distributed system
- Coordinate process restarts across multiple clients
- Monitor process health and handle failures automatically
- Build robust process management into their applications

1.1.2 Prerequisites

- Python 3.10 or higher
- psutil package (required)
- pyzmq package (optional, for ZMQ transport)

Chapter 2

Description

2.1 Description

2.1.1 Introduction

ProcessHub is a process lifecycle management system designed for distributed test automation environments. It provides a centralized hub for starting, stopping, monitoring, and coordinating processes across multiple client panels. The system is built with clean architecture principles, ensuring testability, maintainability, and extensibility.

Key Features

- **Process Lifecycle Management:** Start, stop, and monitor processes with health checking
- **Multi-Client Coordination:** Multiple panels can share and coordinate process ownership
- **Restart Orchestration:** Coordinated restart protocol with acknowledgment from all affected clients
- **Pluggable Transport:** ZMQ, EventBus (RabbitMQ), or in-memory for testing
- **Platform Abstraction:** Windows and Unix process handling
- **Web Dashboard:** Real-time monitoring via browser (FastAPI-based)
- **Logging Integration:** Optional centralized logging (Fluentbit, Telegraf)
- **Fleet Orchestrator:** Coordinate multiple ProcessHub instances across PCs from a central point

Design Philosophy

ProcessHub follows several key design principles:

1. **Separation of Concerns:** Core business logic is isolated from I/O, transport, and UI
2. **Dependency Injection:** All external dependencies are injected, enabling easy testing
3. **Protocol-Oriented:** Components communicate through well-defined protocols (interfaces)
4. **Explicit State Machines:** Complex state transitions use explicit FSM patterns
5. **Immutable Snapshots:** UI receives immutable copies of state, avoiding lock contention

2.1.2 Architecture Overview

ProcessHub is organized into distinct layers, each with specific responsibilities:

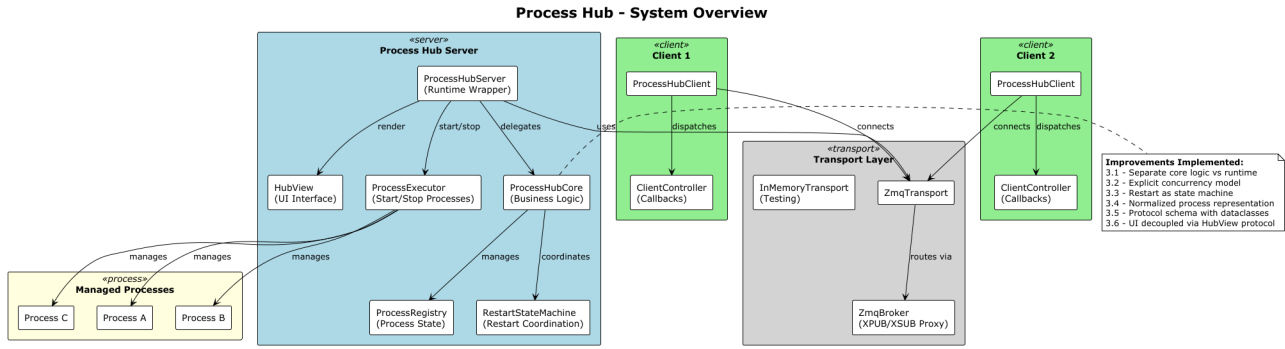


Figure 2.1: Process Hub System Overview

Layer Responsibilities

Core Layer Pure business logic without I/O dependencies. Contains the hub logic, process registry, restart state machine, and data models.

Transport Layer Message passing abstraction. Provides ZMQ for local production, EventBus (RabbitMQ) for distributed environments, and in-memory transport for testing. All implement the same interface.

Runtime Layer Server and client wiring. Thin wrappers that connect core logic to transport and UI. Contains the main event loops.

Process Layer Process execution and platform-specific handling. Abstracts subprocess management for Windows and Unix platforms.

UI Layer View implementations. Console TUI, null view for headless operation, and web dashboard for browser-based monitoring.

Config Layer Configuration loading and validation. Supports JSON/YAML configs with schema validation.

Logging Layer Optional integration with centralized logging servers like Fluentbit or Telegraf.

Fleet Layer Optional overlay for multi-hub coordination. Contains the FleetOrchestrator (central coordinator), HubAgent (sidecar per hub), FleetClient (CI/CD API), HubRegistry (hub tracking), and FleetWebAPI (REST dashboard). Does not modify any other layer.

2.1.3 Component Architecture

The component diagram shows how packages relate to each other:

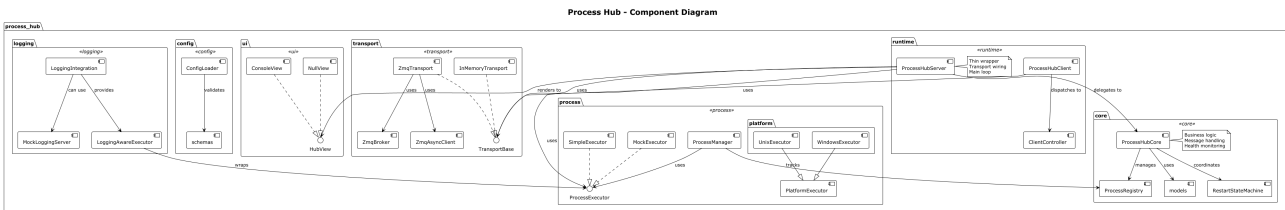


Figure 2.2: Process Hub Component Diagram

ProcessHubCore

The central business logic component implements improvement 3.1 (Separate core logic vs runtime):

- Handles all message processing via `handle_message()` method
- Manages process registry for tracking process state
- Coordinates restart state machine for multi-client restart
- Provides immutable state snapshots for UI rendering

- Has no direct knowledge of ZMQ, TUI, or file I/O

```
# ProcessHubCore is testable without I/O
core = ProcessHubCore(
    process_starter=mock_start,
    process_stopper=mock_stop,
    health_checker=mock_health,
)

# Handle message returns outgoing messages
outgoing = core.handle_message(incoming_msg)

# Get state snapshot (immutable, no locks needed)
snapshot = core.get_state_snapshot()
```

ProcessRegistry

Thread-safe registry implementing improvement 3.4 (Normalized process representation):

- Maintains process information: state, PID, requesters
- Supports multiple panels requesting the same process
- Provides atomic state transitions under single lock
- Returns immutable snapshots for concurrent access

Process States:

REGISTERED Process is known but not yet started

STARTING Process start has been requested, waiting for confirmation

RUNNING Process is confirmed running

STOPPING Process stop has been requested

STOPPED Process has exited normally

DEAD Process was running but died unexpectedly

FAILED Process failed to start

RestartStateMachine

Implements improvement 3.3 (Restart as explicit state machine):

The state machine manages the complete restart coordination flow:

```
IDLE -> NOTIFYING -> AWAITING_ACK -> RESTARTING -> DONE -> IDLE
                        |               |
                        +-----+-----> IDLE
                        FAILED
```

RestartContext holds all restart-related data under a single lock:

- `killed_processes`: List of dead processes detected
- `notified_panels`: All panels that were notified
- `pending_panels`: Panels that haven't acknowledged yet
- `started_at`: Timestamp for timeout detection
- `retry_count`: Number of restart attempts

2.1.4 Communication Patterns

Transport Abstraction

All communication goes through the transport layer, which provides a unified interface:

```
class TransportBase(ABC):
    @abstractmethod
    def start(self) -> None: ...

    @abstractmethod
    def stop(self) -> None: ...

    @abstractmethod
    def send(self, topic: str, message: dict) -> None: ...

    @abstractmethod
    def subscribe(self, topic: str, callback: Callable) -> None: ...
```

ZMQ Transport

Production transport using ZeroMQ PUB/SUB with XPUB/XSUB broker:

- **XPUB/XSUB Broker:** Central message routing with subscription forwarding
- **Topic-based Filtering:** Messages routed by topic prefix
- **Async Client:** Background thread with event loop for non-blocking operation
- **Subscription Propagation:** Delay to ensure subscriptions reach broker

```
# Server with broker
server_transport = ZmqTransport(
    start_broker=True,
    xpub_port=5555,
    xsub_port=5556,
)

# Client connects to broker
client_transport = ZmqTransport(
    start_broker=False,
    xpub_port=5555,
    xsub_port=5556,
)
```

In-Memory Transport

Testing transport for unit tests without network:

- Direct message passing between server and clients
- Synchronous operation for predictable test behavior
- Full protocol compatibility with ZMQ transport

EventBus Transport

RabbitMQ-based transport using the EventBusClient library for enterprise message passing:

- **RabbitMQ Backend:** Robust message broker with persistence and clustering
- **Topic Exchange:** Flexible routing with topic-based message filtering

- **Auto-reconnect:** Automatic reconnection on connection loss
- **JSON Serialization:** Human-readable message format
- **Client Events:** Event-based notifications for connection state changes

Prerequisites:

```
# 1. Install RabbitMQ (Docker recommended)
docker run -d -p 5672:5672 -p 15672:15672 rabbitmq:management

# 2. Install EventBusClient package
git clone https://github.com/test-fullautomation/python-rabbitmq-messagebus.git
cd python-rabbitmq-messagebus && pip install -e .
```

Server Configuration:

```
from ProcessHub.transport.eventbus_transport import EventBusTransport

# Server transport
server_transport = EventBusTransport(
    host="localhost",
    port=5672,
    exchange_name="process_hub",
)
```

Client Configuration:

```
# Client transport (same configuration)
client_transport = EventBusTransport(
    host="localhost",
    port=5672,
    exchange_name="process_hub",
)

# With config file
client_transport = EventBusTransport(config_path="eventbus_config.jsonp")
```

Client Events:

The EventBus transport provides event notifications via the ClientEvent enum:

CONNECTED Client successfully connected to RabbitMQ

DISCONNECTED Client disconnected from RabbitMQ

RECONNECTING Client attempting to reconnect

ERROR An error occurred during communication

```
from ProcessHub.transport.eventbus_transport import ClientEvent

def on_client_event(event: ClientEvent, data: dict):
    if event == ClientEvent.CONNECTED:
        print("Connected to RabbitMQ")
    elif event == ClientEvent.DISCONNECTED:
        print(f"Disconnected: {data.get('reason')}")

transport.register_event_handler(on_client_event)
```

EventBus Config File Format:

```
{
    "host": "localhost",
    "port": 5672,
    "exchange_name": "process_hub",
    "exchange_handler": "TopicExchangeHandler",
    "serializer": "JsonSerializer",
    "auto_reconnect": true,
    "reconnect_delay": 5.0
}
```

2.1.5 Restart Coordination Protocol

When a process dies unexpectedly, ProcessHub coordinates a restart across all affected clients:

Protocol Flow

1. **Detection:** Health check detects dead process
2. **Notification:** Server sends `ProcessRestartNotify` to all affected panels
3. **Acknowledgment:** Each panel saves state and sends `ProcessRestartReady`
4. **Restart:** Once all panels acknowledge (or timeout), server restarts processes
5. **Completion:** Server sends `ProcessRestartDone` to all panels
6. **Resume:** Panels restore state and resume operation

Timeout Handling

If panels don't acknowledge within the configured timeout:

- Restart proceeds anyway to avoid deadlock
- Non-responsive panels receive failure notification
- Timeout is configurable via `restart_timeout` parameter

2.1.6 Protocol Messages

ProcessHub uses typed dataclass messages implementing improvement 3.5 (Protocol schema):

Message Base

All messages inherit from `MessageBase`:

```
@dataclass
class MessageBase:
    version: str = "1.0"
    timestamp: float = field(default_factory=time.time)
    request_id: Optional[str] = None
```

Connection Messages

RegisterConnectionRequest Client registers with server

RegisterConnectionResponse Server confirms registration

UnregisterConnectionRequest Client disconnects

ConnectionInfoRequest Client requests connection list

Process Control Messages

ProcessStartRequest Client requests process start

ProcessStartResponse Server confirms start result

ProcessStopRequest Client requests process stop

ProcessStopResponse Server confirms stop result

Restart Messages

ProcessRestartNotify Server notifies panel of dead processes

ProcessRestartReady Panel acknowledges ready for restart

ProcessRestartDone Server notifies restart completion

2.1.7 Web Dashboard

ProcessHub includes an optional web-based dashboard built with FastAPI:

```
from ProcessHub.ui import WebView

view = WebView(host="0.0.0.0", port=2507)
server = ProcessHubServer(transport=transport, view=view)
server.run()

# Dashboard: http://localhost:2507
# API docs: http://localhost:2507/docs
```

Features

- **Real-time Dashboard:** Auto-refreshing status display
- **Connection Monitoring:** View all connected panels
- **Process Status:** Track process states and PIDs
- **Restart Visualization:** Monitor restart coordination progress
- **REST API:** Programmatic access to hub state
- **Auto-generated Docs:** Swagger UI at /docs, ReDoc at /redoc

API Endpoints

GET / Dashboard HTML page

GET /api/state Complete hub state (connections, processes, restart)

GET /api/connections List of connected panels

GET /api/processes List of managed processes

GET /api/restart Current restart coordination state

GET /api/health Health check endpoint

2.1.8 Logging Integration

ProcessHub supports optional integration with centralized logging servers:

- **Fluentbit**: Log aggregation and forwarding
- **Telegraf**: Metrics collection
- **Custom**: User-defined logging servers via protocol

```
from ProcessHub.logging import LoggingIntegration, LoggingServerType

logging = LoggingIntegration(
    server_type=LoggingServerType.FLUENTBIT,
    paths={
        "src_dir": "/path/to/fluentbit/config",
        "hot_dir": "/path/to/hot/reload",
        "ctrl_dir": "/path/to/control",
    },
    taf_path="/opt/taf",
)
logging.start()

# Get logging config for a process
config = logging.get_process_logging_config("my_process", {
    "central_log": True # Enable for this process
})

# Enhance process args with logging
args = config.to_command_args(taf_path="/opt/taf")
```

LoggingAwareExecutor

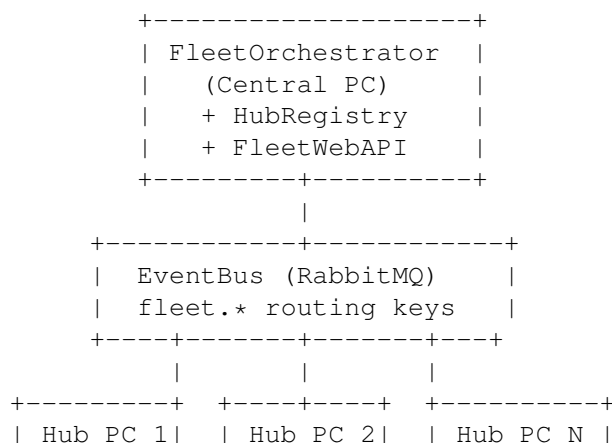
Wraps an executor to automatically add logging configuration:

```
from ProcessHub.logging import LoggingAwareExecutor

executor = LoggingAwareExecutor(
    executor=SimpleExecutor(),
    logging_integration=logging,
    process_configs=process_configs,
)
```

2.1.9 Fleet Orchestrator

ProcessHub includes an optional **Fleet Orchestrator** for coordinating multiple ProcessHub instances across different PCs from a central point. The fleet layer is a pure overlay – it does not modify the core, runtime, or transport modules.



HubAgent	HubAgent	HubAgent
+ Server	+ Server	+ Server
+-----+	+-----+	+-----+

Fleet Components

FleetOrchestrator Central coordinator that discovers hubs via HubAnnounce messages, tracks hub health via heartbeat timeouts, routes commands from FleetClient to target hubs, and provides fleet-wide state snapshots. Runs a periodic `tick()` for health checking.

HubAgent Lightweight sidecar that runs alongside each ProcessHubServer. Sends periodic heartbeats (HubAnnounce) and status reports (HubStatusReport) to the orchestrator. Handles incoming fleet commands (start_process, stop_process, reset, get_status) by delegating to the local server's public API. Uses background daemon threads for heartbeat and status loops.

HubRegistry Thread-safe registry of fleet hubs. Maintains mutable `_HubEntry` objects internally, exposes immutable `HubSnapshot` and `FleetStateSnapshot` externally. Health state transitions: online (heartbeat within timeout/2) → degraded (within timeout) → offline (exceeded timeout).

FleetClient High-level API for CI/CD pipelines, CLI tools, and scripts. Provides single-hub commands (start_processes, stop_processes, reset_hub, get_hub_detail) and batch operations (start_on_all_hubs, stop_all_processes, reset_all_hubs). Supports event callbacks via `on_hub_online()` and `on_hub_offline()`.

FleetWebAPI Optional REST API and HTML dashboard built with FastAPI. Provides endpoints for fleet status, hub listing, hub details, and command routing. HTML dashboard auto-refreshes every 5 seconds. Swagger documentation at `/docs`.

Fleet Protocol Messages

All fleet messages extend `MessageBase` for protocol versioning:

HubAnnounce Hub agent heartbeat and registration (hub_id, hub_name, host, capabilities)

HubRegistered Orchestrator acknowledgement of new hub registration

HubDeregistered Hub removal notification (graceful shutdown or timeout)

HubStatusReport Periodic status from hub agent (process_count, connection_count, processes)

FleetCommand Command from orchestrator to hub agent (command_id, action, params)

FleetCommandResult Result from hub agent back to orchestrator (success, message, data)

Fleet Topics

Fleet topics use the `FLEET_` prefix to separate from local hub topics:

```
class FleetTopics(str, Enum):
    HUB_ANNOUNCE      = "FLEET_HUB_ANNOUNCE"
    HUB_REGISTERED    = "FLEET_HUB_REGISTERED"
    HUB_DEREGISTER    = "FLEET_HUB_DEREGISTER"
    HUB_STATUS_REPORT = "FLEET_HUB_STATUS_REPORT"
    FLEET_COMMAND      = "FLEET_COMMAND"
    FLEET_COMMAND_RESULT = "FLEET_COMMAND_RESULT"
    HUB_ONLINE         = "FLEET_HUB_ONLINE"
    HUB_OFFLINE        = "FLEET_HUB_OFFLINE"
```

Fleet Quick Start

```
from ProcessHub.fleet import FleetOrchestrator, HubAgent, FleetClient
from ProcessHub.transport.inmemory_transport import InMemoryTransport

# Central PC: Start orchestrator
fleet_transport = InMemoryTransport()
```

```

fleet_transport.start()

orchestrator = FleetOrchestrator(
    transport=fleet_transport, health_timeout=30.0
)
orchestrator.start()

# Hub PC: Attach agent to existing server
agent = HubAgent(
    server=my_server,
    transport=fleet_transport,
    hub_id="bench-1",
    hub_name="HIL Bench 1",
)
agent.start()

# CI/CD: Use FleetClient
client = FleetClient(
    transport=fleet_transport, orchestrator=orchestrator
)

# Check fleet health
snapshot = client.get_fleet_status()
print(f"Online: {snapshot.online_hubs}/{snapshot.total_hubs}")

# Start processes on a specific hub
client.start_processes("bench-1", ["ecu_simulator"])

# Batch operations across all online hubs
client.start_on_all_hubs(["data_logger"])
client.reset_all_hubs()

```

Fleet Web Dashboard

```

from ProcessHub.fleet.web_api import FleetWebAPI

web = FleetWebAPI(
    orchestrator=orchestrator, host="0.0.0.0", port=2510
)
web.start()
# Dashboard: http://localhost:2510
# Swagger:    http://localhost:2510/docs

```

Fleet REST API Endpoints

GET /api/fleet/status Fleet-wide status snapshot (total_hubs, online_hubs, hubs list)

GET /api/fleet/hubs List all registered hubs

GET /api/fleet/hubs/{hub_id} Single hub details

POST /api/fleet/command Send a command to a hub (hub_id, action, params)

POST /api/fleet/hubs/{hub_id}/start Start processes on a hub

POST /api/fleet/hubs/{hub_id}/stop Stop processes on a hub

POST /api/fleet/hubs/{hub_id}/reset Reset a hub

GET / HTML fleet monitoring dashboard (auto-refreshes every 5 seconds)

Fleet Installation

```
# Install with fleet support
pip install python-process-hub[fleet]

# Fleet web API requires FastAPI and uvicorn
pip install fastapi uvicorn
```

2.1.10 Installation

From PyPI

```
# Basic installation
pip install python-process-hub

# With ZMQ support
pip install python-process-hub[zmq]

# With web dashboard
pip install python-process-hub[web]

# With fleet orchestrator
pip install python-process-hub[fleet]

# Development dependencies
pip install python-process-hub[dev]

# All optional dependencies
pip install python-process-hub[zmq,web,fleet,dev]
```

From Source

```
# Clone repository
git clone https://github.com/test-fullautomation/python-process-hub.git
cd python-process-hub

# Install in development mode
python setup.py develop

# Or install normally
python setup.py install
```

2.1.11 Quick Start

Server Example

```
import sys
from ProcessHub.runtime.server import ProcessHubServer
from ProcessHub.transport.zmq_transport import ZmqTransport
from ProcessHub.ui.console_view import ConsoleView
from ProcessHub.process.executor import SimpleExecutor

# Create transport with broker
transport = ZmqTransport(start_broker=True)

# Create view and executor
view = ConsoleView()
executor = SimpleExecutor()

# Define process configurations
process_config = {
```

```

    "system_manager": {
        "script": sys.executable,
        "args": ["-c", "import time; time.sleep(3600)"],
        "wait_time": 2.0,
    },
},
}

# Create and run server
server = ProcessHubServer(
    transport=transport,
    view=view,
    executor=executor,
    process_config=process_config,
)
server.run() # Blocks until Ctrl+C

```

Client Example

```

from ProcessHub.runtime.client import ProcessHubClient
from ProcessHub.transport.zmq_transport import ZmqTransport

# Connect to existing broker
transport = ZmqTransport(start_broker=False)
client = ProcessHubClient(transport=transport)
client.start()

# Register with server
client.register_connection(session_id="my_session")
client.wait_for_registration(timeout=10.0)

# Start processes
client.request_process_start(["system_manager"])

# Handle restart notifications via callbacks
def on_restart_notify(killed_processes):
    print(f"Processes died: {killed_processes}")
    # Save state, prepare for restart
    client.notify_restart_ready()

client.controller.connect("restart_notify", on_restart_notify)

```

2.1.12 Testing

ProcessHub is designed for testability using the in-memory transport:

```

from ProcessHub.transport.inmemory_transport import InMemoryTransport
from ProcessHub.process.executor import MockExecutor

# Create in-memory transport (no ZMQ needed)
transport = InMemoryTransport()

# Create mock executor (no real processes)
executor = MockExecutor(process_configs)

# Test core logic without I/O
server = ProcessHubServer(
    transport=transport,
    executor=executor,
    view=NullView(),
)

```

Running Tests

```
# Run all tests
pytest

# With coverage
pytest --cov=ProcessHub

# Specific test file
pytest tests/test_core.py -v
```

2.1.13 Extensibility

ProcessHub provides several extension points:

Custom Transport

Implement TransportBase to add new transport mechanisms:

```
from ProcessHub.transport.base import TransportBase

class MyTransport(TransportBase):
    def start(self) -> None: ...
    def stop(self) -> None: ...
    def send(self, topic: str, message: dict) -> None: ...
    def subscribe(self, topic: str, callback: Callable) -> None: ...
```

Custom Executor

Implement ProcessExecutorProtocol for custom process handling:

```
class MyExecutor:
    def start_process(self, name: str) -> tuple[bool, str, int]: ...
    def stop_process(self, name: str, force: bool = False) -> bool: ...
    def is_running(self, name: str) -> bool: ...
```

Custom View

Implement HubView protocol for custom UI:

```
from ProcessHub.ui.base import HubViewBase

class MyView(HubViewBase):
    def start(self) -> None: ...
    def stop(self) -> None: ...
    def render(self, snapshot: HubStateSnapshot) -> None: ...
```

Custom Logging Server

Implement LoggingServerProtocol or extend LoggingServerBase:

```
from ProcessHub.logging import LoggingServerBase

class MyLoggingServer(LoggingServerBase):
    def allocate(self, count: int) -> tuple[str, list[int]]: ...
    def deallocate(self) -> None: ...
```

2.1.14 Design Improvements

ProcessHub implements several architectural improvements over the original ta-framework process_hub:

- 3.1 - Separate Core Logic vs Runtime** ProcessHubCore contains pure business logic testable without I/O. ProcessHubServer is a thin wrapper for transport wiring.
- 3.2 - Explicit Concurrency Model** Each data structure has a single lock. ProcessRegistry and RestartStateMachine use their own locks. HubStateSnapshot provides immutable copies for UI.
- 3.3 - Restart as State Machine** RestartStateMachine with explicit states (IDLE, NOTIFYING, AWAITING_ACK, RESTARTING, DONE, FAILED). All restart state in RestartContext under single lock.
- 3.4 - Normalized Process Representation** ProcessState enum replaces inconsistent status field. ProcessInfo dataclass with explicit fields replaces dict-based shared_info.
- 3.5 - Protocol Schema** Typed dataclass messages with versioning. Each message type has explicit fields for validation and documentation.
- 3.6 - UI Decoupled** HubView protocol for rendering. ConsoleView, NullView, WebView implementations. State passed as immutable snapshot.

2.1.15 Diagrams Reference

The following PlantUML diagrams are available in the docs/diagrams/ folder:

- overview.puml** System overview showing server, clients, and processes
- component.puml** Component diagram with package dependencies
- class_core.puml** Class diagram for core package
- class_transport.puml** Class diagram for transport package
- class_runtime.puml** Class diagram for runtime package
- class_process.puml** Class diagram for process package
- class_ui.puml** Class diagram for UI package
- sequence_registration.puml** Client registration sequence
- sequence_process_start.puml** Process start sequence
- sequence_restart_flow.puml** Restart coordination sequence
- state_restart.puml** Restart state machine
- fleet_overview.puml** Fleet system overview with orchestrator, hub PCs, and clients
- fleet_component.puml** Fleet component diagram with module dependencies
- class_fleet.puml** Fleet models, registry, orchestrator, agent, and client classes
- sequence_fleet_discovery.puml** Hub discovery, heartbeat, and deregistration sequence
- sequence_fleet_command.puml** Command routing from client through orchestrator to hub
- state_hub_health.puml** Hub health state machine (online/degraded/offline transitions)

To generate PNG images from PlantUML files:

```
# Using PlantUML CLI
java -jar plantuml.jar docs/diagrams/*.puml

# Using Docker
docker run -v $(pwd)/diagrams:/data plantuml/plantuml *.puml
```

Restart State Machine (Improvement 3.3)

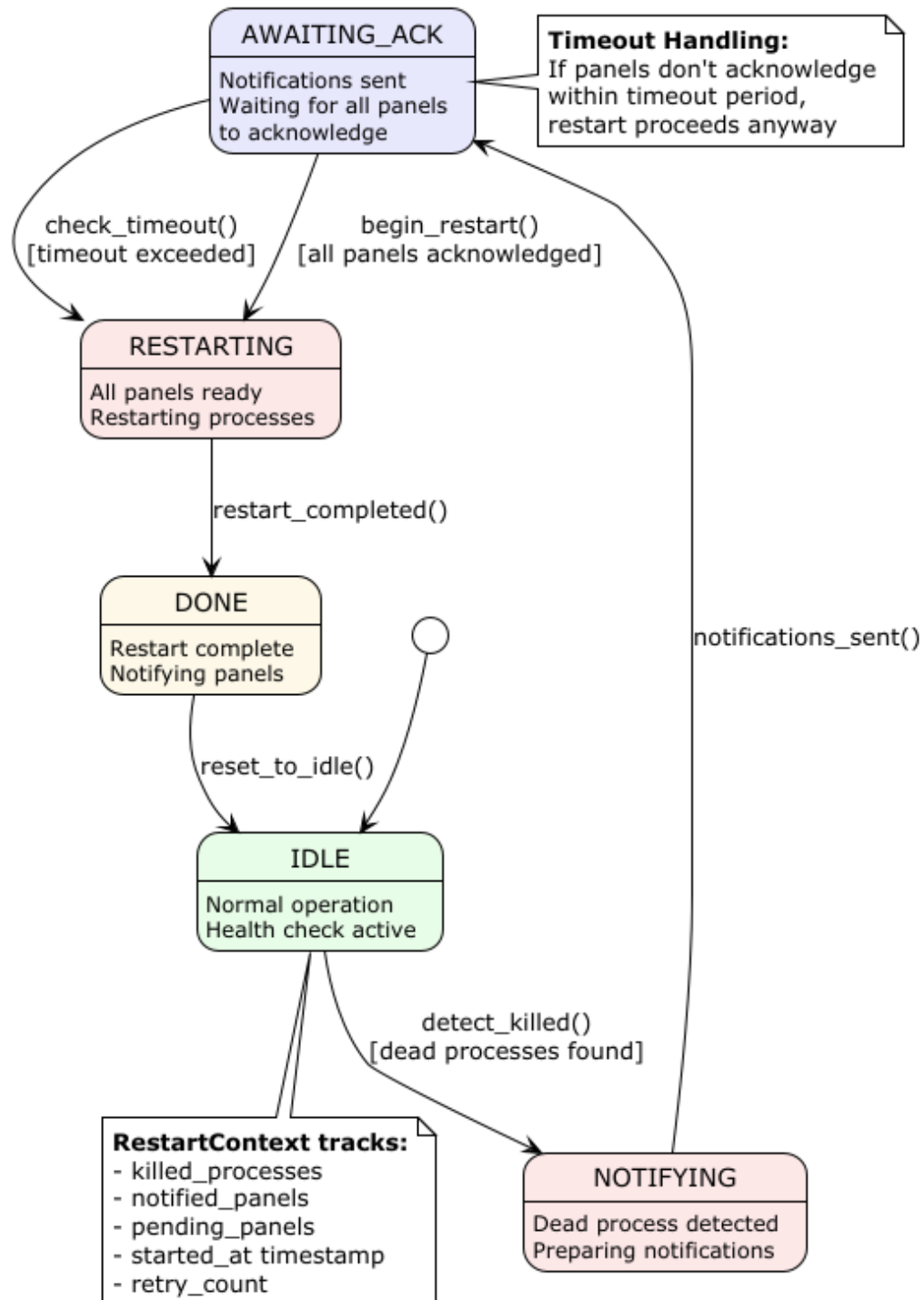


Figure 2.3: Restart State Machine

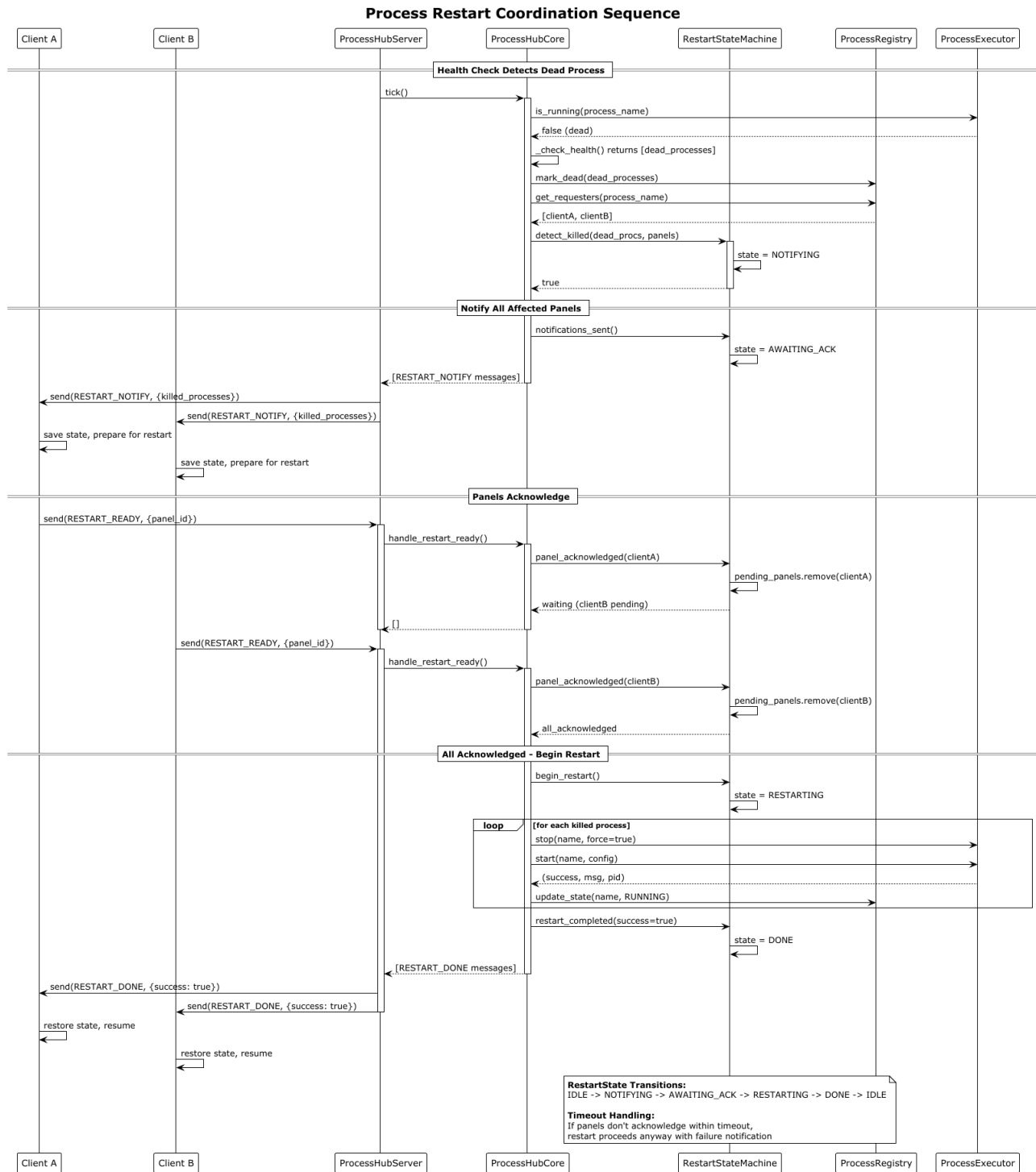


Figure 2.4: Restart Coordination Sequence

Chapter 3

`__init__.py`

Python Process Hub - Process lifecycle management with ZMQ-based coordination.

This package provides a robust process management hub that:

- Manages process lifecycle (start, stop, restart)
- Coordinates restarts across multiple clients
- Provides health monitoring and automatic recovery
- Supports pluggable transports (ZMQ, in-memory for testing)

Example usage (Server):

```
from ProcessHub import ProcessHubServer
from ProcessHub.transport import InMemoryTransport
from ProcessHub.ui import ConsoleView
from ProcessHub.process import SimpleExecutor
transport = InMemoryTransport()
view = ConsoleView()
executor = SimpleExecutor()
server = ProcessHubServer(
    transport=transport,
    view=view,
    executor=executor,
)
server.run()
```

Example usage (Client):

```
from ProcessHub import ProcessHubClient
from ProcessHub.transport import InMemoryTransport
transport = InMemoryTransport()
client = ProcessHubClient(transport=transport, panel_id="my_panel")
client.start()
client.register_connection(session_id="my_session")
client.request_process_start(["my_process"])
```

Chapter 4

`__init__.py`

Configuration loading and schemas.

Provides utilities for loading process configuration from various sources and typed schemas for validation.

Loader classes:

- `JsonConfigLoader`: Load from JSON files with env var expansion
- `JsonPreprocessorLoader`: Compatibility with ta-framework's `JsonPreprocessor`
- `ProcessConfigLoader`: High-level process configuration loader

Schema classes:

- `ProcessConfig`: Single process configuration
- `ProcessesConfig`: Collection of processes
- `ServerConfig`: Server configuration
- `ClientConfig`: Client configuration
- `TransportConfig`: Transport configuration
- `LoggingServerConfig`: Logging server configuration

Chapter 5

loader.py

Configuration loader.

This module provides utilities for loading and processing process configuration from various sources (JSON, JSONP, dict).

Replaces the dependency on `JsonPreprocessor.CJsonPreprocessor` with a flexible, pluggable loader system.

5.1 Function: `expand_vars`

Expand environment variables in a string.

Supports formats:

- `${VAR}` or `$VAR` - standard env var
- `%VAR%` - Windows style

Arguments:

- `value`
/ *Condition*: required / *Type*: str /
String to expand.
- `env`
/ *Condition*: optional / *Type*: dict / *Default*: None /
Environment dict (uses `os.environ` if None).

Returns:

/ *Type*: str /

Expanded string.

5.2 Function: `fully_expand_vars`

Fully expand nested environment variables.

Expands repeatedly until no more changes occur.

5.3 Function: `expand_and_quote`

Expand variables and quote if contains spaces.

Arguments:

- `value`
/ *Condition*: required / *Type*: str /
String to process.
- `env`
/ *Condition*: optional / *Type*: dict / *Default*: None /
Environment dict.

Returns:/ *Type*: str /

Expanded and possibly quoted string.

5.4 Function: compute_restart_sequence

Compute the restart sequence for a process.

Determines which processes need to be restarted based on dependencies.

Arguments:

- `config`
/ *Condition*: required / *Type*: dict /
Process configuration dict.
- `process_name`
/ *Condition*: required / *Type*: str /
Name of process that died.

Returns:/ *Type*: list[str] /

List of process names in restart order.

5.5 Class: ConfigLoader

Imported by:

```
from ProcessHub.config.loader import ConfigLoader
```

Abstract configuration loader. processhub-config-loader-configloader-load -----

Load configuration from source.

Arguments:

- `source`
/ *Condition*: required / *Type*: Union[str, Path, dict] /
File path, URL, or dict.

Returns:/ *Type*: dict /

Configuration dict.

5.6 Class: JsonConfigLoader

Imported by:

```
from ProcessHub.config.loader import JsonConfigLoader
```

JSON configuration loader.

Loads configuration from JSON files with optional environment variable expansion.

5.6.1 Method: load

Load configuration from JSON file or dict.

5.7 Class: JsonPreprocessorLoader

Imported by:

```
from ProcessHub.config.loader import JsonPreprocessorLoader
```

JSON Preprocessor loader (compatibility with ta-framework).

Uses JsonPreprocessor.CJsonPreprocessor if available, falls back to standard JSON loading.

5.7.1 Method: load

Load configuration using JsonPreprocessor or fallback.

5.8 Class: ProcessConfigLoader

Imported by:

```
from ProcessHub.config.loader import ProcessConfigLoader
```

Process configuration loader.

Loads process definitions and provides utilities for accessing process-specific configuration.

Usage:

```
loader = ProcessConfigLoader()
loader.load_file("${TAF_PATH}/project/processes/config.jsonp")
config = loader.get_process_config("system_manager")
args = loader.get_process_arguments("system_manager")
```

5.8.1 Method: config

Get the full configuration dict.

5.8.2 Method: load_file

Load configuration from file.

Arguments:

- path
/ *Condition*: required / *Type*: Union[str, Path] /
Path to configuration file.

Returns:

/ *Type*: dict /
Loaded configuration.

5.8.3 Method: load_dict

Load configuration from dict.

Arguments:

- `config`
/ *Condition*: required / *Type*: dict /
Configuration dict.

Returns:

/ *Type*: dict /
The configuration.

5.8.4 Method: get_process_names

Get all process names.

5.8.5 Method: get_process_config

Get configuration for a specific process.

Arguments:

- `name`
/ *Condition*: required / *Type*: str /
Process name.

Returns:

/ *Type*: Optional[dict] /
Process configuration or None.

5.8.6 Method: has_process

Check if process is defined in config.

5.8.7 Method: get_process_arguments

Get process arguments for starting.

Mirrors ProcessControl.get_process_arguments().

Arguments:

- `name`
/ *Condition*: required / *Type*: str /
Process name.
- `logging_config`
/ *Condition*: optional / *Type*: dict / *Default*: None /
Optional logging configuration dict with keys: `config_dir`, `host`, `port`.

Returns:

/ *Type*: Optional[tuple] /
Tuple of (name, script, args, logging_conf, logging_host, logging_port) or None if process not found.

5.8.8 Method: `get_executor_config`

Get configuration formatted for executor.

Arguments:

- `name`
/ *Condition*: required / *Type*: str /
Process name.

Returns:

/ *Type*: dict /
Dict suitable for `ProcessExecutor.start()`.

Chapter 6

schemas.py

Configuration schemas using dataclasses.

This module defines typed configuration schemas for:

- Process definitions
- Server configuration
- Client configuration

These can optionally be used with Pydantic for validation if pydantic is installed, otherwise they work as plain dataclasses.

6.1 Function: `validate_process_config`

Validate and create `ProcessConfig` from dict.

Arguments:

- `config`
/ *Condition*: required / *Type*: dict /
Raw configuration dict.

Returns:

/ *Type*: `ProcessConfig` /
Validated `ProcessConfig`.

Raises:

- `ValueError`: If validation fails.

6.2 Function: `validate_server_config`

Validate and create `ServerConfig` from dict.

Arguments:

- `config`
/ *Condition*: required / *Type*: dict /
Raw configuration dict.

Returns:

/ *Type*: `ServerConfig` /
Validated `ServerConfig`.

6.3 Function: validate_client_config

Validate and create ClientConfig from dict.

Arguments:

- config
/ Condition: required / Type: dict /
Raw configuration dict.

Returns:

/ Type: ClientConfig /
Validated ClientConfig.

6.4 Class: ProcessConfig

Imported by:

```
from ProcessHub.config.schemas import ProcessConfig
```

Configuration for a single process.

Attributes:

process_name: Display name / window title
script: Path to script or executable
args: Command line arguments
wait_time: Time to wait after starting (seconds)
exec_type: Execution type ("python" or "external")
central_log: Whether to use central logging
dependencies: List of dependent process names
env: Additional environment variables
working_dir: Working directory for process
enable: Whether the process is enabled for ProcessHub
description: Brief description of what the process does
mandatory: Whether the process is mandatory for testing
warning_on_deselect: Warning message when deselecting the process

6.4.1 Method: to_executor_config

Convert to executor configuration dict.

6.5 Class: ProcessesConfig

Imported by:

```
from ProcessHub.config.schemas import ProcessesConfig
```

Collection of process configurations.

Attributes:

processes: Dict of process name to configuration

6.5.1 Method: get

Get process configuration by name.

6.5.2 Method: names

Get all process names.

6.5.3 Method: from_dict

Create from raw dict (e.g., loaded from JSON).

6.6 Class: TransportConfig

Imported by:

```
from ProcessHub.config.schemas import TransportConfig
```

Transport configuration.

Attributes:

- type: Transport type ("zmq" or "inmemory")
- start_broker: Whether to start broker (ZMQ only)
- xpub_port: X PUB port for subscribers
- xsub_port: X SUB port for publishers
- bind_address: Address to bind to
- connect_address: Address to connect to

6.7 Class: LoggingServerConfig

Imported by:

```
from ProcessHub.config.schemas import LoggingServerConfig
```

Logging server configuration.

Attributes:

- type: Server type ("fluentbit", "telegraf", or "mock")
- src_dir: Source directory for configs
- hot_dir: Hot reload directory
- influx_dir: InfluxDB directory
- ctrl_dir: Control directory

6.8 Class: ServerConfig

Imported by:

```
from ProcessHub.config.schemas import ServerConfig
```

Process Hub Server configuration.

Attributes: transport: Transport configuration
logging: Logging server configuration
refresh_interval: Main loop refresh interval (seconds)
restart_timeout: Restart coordination timeout (seconds)
process_config_path: Path to process configuration file

6.8.1 Method: from_dict

Create from raw dict.

6.9 Class: ClientConfig

Imported by:

```
from ProcessHub.config.schemas import ClientConfig
```

Process Hub Client configuration.

Attributes:

transport: Transport configuration
panel_id: Panel identifier (auto-generated if None)
default_timeout: Default request timeout (seconds)
auto_reconnect: Whether to auto-reconnect on disconnect
reconnect_interval: Time between reconnect attempts (seconds)

6.9.1 Method: from_dict

Create from raw dict.

Chapter 7

`__init__.py`

Core business logic - no I/O dependencies.

This module contains the pure business logic for process hub:

- Models: Data structures and protocol messages
- ProcessRegistry: Thread-safe process state management
- RestartStateMachine: Explicit restart state machine
- ProcessHubCore: Core hub logic (handles all business operations)

Chapter 8

hub_core.py

ProcessHubCore - pure business logic, no I/O.

This module implements improvement 3.1:

- Separate core logic vs. runtime
- Owns ProcessRegistry, ConnectionRegistry, RestartStateMachine
- Exposes handle_message() -> returns outgoing messages
- Has no direct knowledge of ZMQ, no TUI

Benefits:

- Immediately improves testability
- Makes it possible to embed Process Hub inside other processes/transport

8.1 Class: MessageType

Imported by:

```
from ProcessHub.core.hub_core import MessageType
```

MessageType: Outgoing message types for transport routing.

8.2 Class: OutgoingMessage

Imported by:

```
from ProcessHub.core.hub_core import OutgoingMessage
```

OutgoingMessage: Wrapper for outgoing messages.

Used by runtime to route messages to transport layer.

8.3 Class: ProcessExecutorProtocol

Imported by:

```
from ProcessHub.core.hub_core import ProcessExecutorProtocol
```

ProcessExecutorProtocol: Protocol for process executor callbacks.

8.3.1 Method: start_process

Start a process.

Arguments:

- name
/ *Condition*: required / *Type*: str /
The name of the process to start.

Returns:

/ *Type*: tuple[bool, str, Optional[int]] /
A tuple of (success, message, pid).

8.3.2 Method: stop_process

Stop a process.

Arguments:

- name
/ *Condition*: required / *Type*: str /
The name of the process to stop.
- force
/ *Condition*: optional / *Type*: bool / *Default*: False /
If True, force stop the process without graceful shutdown.

Returns:

/ *Type*: bool /
True if the process was stopped successfully, False otherwise.

8.3.3 Method: is_running

Check if process is running.

Arguments:

- name
/ *Condition*: required / *Type*: str /
The name of the process to check.

Returns:

/ *Type*: bool /
True if the process is running, False otherwise.

8.4 Class: ConnectionInfo

Imported by:

```
from ProcessHub.core.hub_core import ConnectionInfo
```

ConnectionInfo: Connection metadata for a panel.

8.5 Class: ProcessHubCore

Imported by:

```
from ProcessHub.core.hub_core import ProcessHubCore
```

ProcessHubCore: Core hub logic - testable without I/O.

This class contains all business logic for process management:

- Connection registration/unregistration
- Process start/stop requests
- Restart coordination
- Health check detection

It does NOT know about: - ZMQ or any transport

- TUI or any display
- File I/O or logging configuration

Usage:

```
# Create with executor callbacks
core = ProcessHubCore(
    process_starter=my_start_func,
    process_stopper=my_stop_func,
    health_checker=my_health_func,
)
# Handle incoming message
outgoing = core.handle_register(request)
for msg in outgoing:
    transport.send(msg.msg_type.value, msg.payload)

# Periodic tick (health check)
outgoing = core.tick()
for msg in outgoing:
    transport.send(msg.msg_type.value, msg.payload)

# Get state for UI
snapshot = core.get_state_snapshot()
```

8.5.1 Method: get_state_snapshot

Get immutable state snapshot for UI.

Returns frozen dataclass - no locks needed by caller. UI can safely read this without synchronization.

Returns:

/ *Type*: HubStateSnapshot /

An immutable snapshot of the current hub state including connections, processes, and restart state.

8.5.2 Method: `handle_register`

Handle connection registration.

Registers a panel and updates session groupings.

Arguments:

- `req`
/ *Condition*: required / *Type*: `RegisterConnectionRequest` /
The registration request containing `panel_id` and `session_id`.

Returns:

/ *Type*: `list[OutgoingMessage]` /
List of outgoing messages to send (registration response).

8.5.3 Method: `handle_unregister`

Handle connection unregistration.

Removes panel and cleans up any orphaned processes.

Arguments:

- `req`
/ *Condition*: required / *Type*: `UnregisterConnectionRequest` /
The unregistration request containing `panel_id`.

Returns:

/ *Type*: `list[OutgoingMessage]` /
List of outgoing messages to send (unregistration response).

8.5.4 Method: `handle_connection_info`

Handle connection info request.

Arguments:

- `req`
/ *Condition*: required / *Type*: `ConnectionInfoRequest` /
The request containing `panel_id`.

Returns:

/ *Type*: `list[OutgoingMessage]` /
List of outgoing messages containing connection information.

8.5.5 Method: `is_registered`

Check if panel is registered.

Arguments:

- `panel_id`
/ *Condition*: required / *Type*: `str` /
The panel ID to check.

Returns:

/ *Type*: `bool` /
True if the panel is registered, False otherwise.

8.5.6 Method: `handle_start_request`

Handle process start request.

Starts requested processes sequentially. On failure, rolls back all started processes.

Arguments:

- `req`
/ *Condition*: required / *Type*: `ProcessStartRequest` /
The start request containing `panel_id` and `process_list`.

Returns:

/ *Type*: `list[OutgoingMessage]` /
List of outgoing messages (start response with success/failure info).

8.5.7 Method: `handle_stop_request`

Handle process stop request.

Stops processes only if no other panels need them, unless `force=True`.

Arguments:

- `req`
/ *Condition*: required / *Type*: `ProcessStopRequest` /
The stop request containing `panel_id`, `process_list`, and `force` flag.

Returns:

/ *Type*: `list[OutgoingMessage]` /
List of outgoing messages (stop response).

8.5.8 Method: `handle_restart_ready`

Handle panel restart acknowledgment.

When all panels have acknowledged, performs the restart.

Arguments:

- `req`
/ *Condition*: required / *Type*: `ProcessRestartReady` /
The restart ready message from a panel.

Returns:

/ *Type*: `list[OutgoingMessage]` /
List of outgoing messages (restart done notifications if all panels acknowledged).

8.5.9 Method: `tick`

Periodic tick - run health check, detect killed processes.

Should be called periodically (e.g., every 0.5 seconds). Returns outgoing messages (restart notifications if needed).

Returns:

/ *Type*: `list[OutgoingMessage]` /
List of outgoing messages. Empty if no dead processes detected, otherwise restart notifications.

8.5.10 Method: `get_connection_count`

Get number of registered connections.

Returns:

/ Type: int /

The number of currently registered panel connections.

8.5.11 Method: `get_process_count`

Get number of registered processes.

Returns:

/ Type: int /

The number of currently registered processes.

8.5.12 Method: `register_admin_process`

Register a process started by admin (not via client request).

This allows processes started via the admin API to appear in the dashboard and be tracked by the registry.

Arguments:

- `name`

/ Condition: required / Type: str /

The process name.

- `pid`

/ Condition: required / Type: int /

The process ID.

Returns:

/ Type: bool /

True if registered successfully, False if already exists.

8.5.13 Method: `unregister_admin_process`

Unregister an admin-started process.

Called when an admin-started process is stopped.

Arguments:

- `name`

/ Condition: required / Type: str /

The process name.

Returns:

/ Type: bool /

True if unregistered successfully.

8.5.14 Method: reset

Reset the hub to a fresh state.

Stops all processes, clears all connections, and resets the restart state machine. Unlike `shutdown()`, this does not stop the server - it just clears state. Returns notification messages to send to clients before they are disconnected.

Returns:

/ Type: tuple[bool, str, list[OutgoingMessage]] /

A tuple of (success, message, outgoing_messages) where outgoing_messages contains shutdown notifications for all connected clients.

8.5.15 Method: shutdown

Shutdown the hub.

Stops all processes and clears state. Should be called when the server is shutting down. Returns notification messages to send to clients before shutdown.

Returns:

/ Type: list[OutgoingMessage] /

List of shutdown notifications for all connected clients.

Chapter 9

models.py

Data models for Process Hub - normalized representations and protocol schema.

This module implements:

- 3.4: Normalized process representation (ProcessState enum, ProcessInfo dataclass)
- 3.5: Protocol schema (typed message dataclasses with versioning)
- 3.2: Immutable snapshots for UI (frozen dataclasses)

9.1 Class: ProcessState

Imported by:

```
from ProcessHub.core.models import ProcessState
```

ProcessState: Explicit process lifecycle states.

Replaces the inconsistent status field that was:

- Initialized as string "None"
- Then set to boolean True/False

Now all states are explicit enum values.

9.2 Class: ProcessInfo

Imported by:

```
from ProcessHub.core.models import ProcessInfo
```

ProcessInfo: Normalized process representation.

Replaces the dict-based shared_info with explicit fields:

- name: process identifier (was dict key)
- state: ProcessState enum (was "status": "None"/True/False)
- pid: process ID (was "pid": 0)
- requesters: set of panel IDs (was "requester": list)
- error_message: failure reason (new field)
- metadata: extensible data (new field)

9.2.1 Method: add_requester

Add a requester to this process.

Arguments:

- `requester_id`
/ *Condition*: required / *Type*: str /
The panel ID of the requester to add.

9.2.2 Method: remove_requester

Remove requester from process.

Arguments:

- `requester_id`
/ *Condition*: required / *Type*: str /
The panel ID of the requester to remove.

Returns:

/ *Type*: bool /
True if no requesters remain (process can be stopped), False otherwise.

9.2.3 Method: is_active

Check if process is in an active state (STARTING or RUNNING).

Returns:

/ *Type*: bool /
True if the process is starting or running, False otherwise.

9.2.4 Method: to_dict

Convert to dict for backward compatibility / UI.

Maintains similar structure to old `shared_info` format.

Returns:

/ *Type*: dict /
A dictionary with keys: `pid`, `requester`, `status`, `error`.

9.3 Class: MessageBase

Imported by:

```
from ProcessHub.core.models import MessageBase
```

MessageBase: Base class for all protocol messages.

Adds:

- `version`: for protocol evolution
- `timestamp`: for debugging/ordering
- `request_id`: for request/response correlation

9.4 Class: RegisterConnectionRequest

Imported by:

```
from ProcessHub.core.models import RegisterConnectionRequest
```

RegisterConnectionRequest: Client -> Server message to register a panel connection.

9.5 Class: RegisterConnectionResponse

Imported by:

```
from ProcessHub.core.models import RegisterConnectionResponse
```

RegisterConnectionResponse: Server -> Client message with registration result.

9.6 Class: UnregisterConnectionRequest

Imported by:

```
from ProcessHub.core.models import UnregisterConnectionRequest
```

UnregisterConnectionRequest: Client -> Server message to unregister a panel connection.

9.7 Class: UnregisterConnectionResponse

Imported by:

```
from ProcessHub.core.models import UnregisterConnectionResponse
```

UnregisterConnectionResponse: Server -> Client message with unregistration result.

9.8 Class: ConnectionInfoRequest

Imported by:

```
from ProcessHub.core.models import ConnectionInfoRequest
```

ConnectionInfoRequest: Client -> Server message to request connection info.

9.9 Class: ConnectionInfoResponse

Imported by:

```
from ProcessHub.core.models import ConnectionInfoResponse
```

ConnectionInfoResponse: Server -> Client message with connection info.

9.10 Class: ProcessStartRequest

Imported by:

```
from ProcessHub.core.models import ProcessStartRequest
```

ProcessStartRequest: Client -> Server message to start process(es).

Fields: - panelId: requesting panel

- process_list: processes to start
- timeout_per_process: max time to wait per process

9.11 Class: ProcessStartResponse

Imported by:

```
from ProcessHub.core.models import ProcessStartResponse
```

ProcessStartResponse: Server -> Client message with start result.

Replaces old format:

```
{"panelId": ..., "status": bool, "failed_proc": [], "failed_msg": ""}
```

With explicit fields and consistent naming.

9.12 Class: ProcessStopRequest

Imported by:

```
from ProcessHub.core.models import ProcessStopRequest
```

ProcessStopRequest: Client -> Server message to stop process(es).

9.13 Class: ProcessStopResponse

Imported by:

```
from ProcessHub.core.models import ProcessStopResponse
```

ProcessStopResponse: Server -> Client message with stop result.

Provides detailed feedback on what happened to each process:

- stopped - processes that were actually terminated
- still_in_use - processes where only the requester was removed (other clients still need them)
- failed - processes that failed to stop

9.14 Class: ProcessRestartNotify

Imported by:

```
from ProcessHub.core.models import ProcessRestartNotify
```

ProcessRestartNotify: Server -> Client message indicating process(es) died and restart is needed.

Sent to all affected panels when health check detects dead processes.

9.15 Class: ProcessRestartReady

Imported by:

```
from ProcessHub.core.models import ProcessRestartReady
```

ProcessRestartReady: Client -> Server message indicating panel is ready for restart.

Sent by client after receiving ProcessRestartNotify and completing any necessary cleanup.

9.16 Class: ProcessRestartDone

Imported by:

```
from ProcessHub.core.models import ProcessRestartDone
```

ProcessRestartDone: Server -> Client message indicating restart completed.

Sent to all affected panels after restart sequence completes.

9.17 Class: ServerShutdownNotify

Imported by:

```
from ProcessHub.core.models import ServerShutdownNotify
```

ServerShutdownNotify: Server -> Client message indicating server is shutting down or resetting.

Sent to all connected panels before server shutdown or reset. Clients should handle this by cleaning up their state and potentially reconnecting.

9.18 Class: ConnectionSnapshot

Imported by:

```
from ProcessHub.core.models import ConnectionSnapshot
```

ConnectionSnapshot: Immutable snapshot of a connection.

Used by UI to display connection state without holding locks.

9.19 Class: ProcessSnapshot

Imported by:

```
from ProcessHub.core.models import ProcessSnapshot
```

ProcessSnapshot: Immutable snapshot of a process.

Used by UI to display process state without holding locks.

9.20 Class: HubStateSnapshot

Imported by:

```
from ProcessHub.core.models import HubStateSnapshot
```

HubStateSnapshot: Immutable snapshot of hub state for UI.

Benefits:

- Core updates internal mutable state
- `get_state()` returns a fresh snapshot (copy)
- UI can read snapshot without locks
- No race conditions between UI and core

This replaces the pattern of:

with `shared_info_lock`:

```
proc_data = copy.deepcopy(proc_info)
con_data = copy.deepcopy(self.connections)
```

Chapter 10

process_registry.py

Process registry - centralized process state with explicit concurrency.

This module implements improvement 3.2:

- Single lock protects all process state
- `get_snapshot()` returns immutable copy for UI
- Clear documentation of what data is protected

Replaces the scattered state in `ProcessManager`:

- `self.process_map`
- `self.process_requester`
- `self.shared_info`
- `self.shared_info_lock`

10.1 Class: ProcessRegistry

Imported by:

```
from ProcessHub.core.process_registry import ProcessRegistry
```

`ProcessRegistry`: Thread-safe process registry.

Concurrency model (3.2):

- Single `_lock` protects all process state
- All public methods acquire lock
- `get_snapshot()` returns immutable data for UI (no external locking needed)

Usage:

```
registry = ProcessRegistry()
# Register process
registry.register("my_process", requester="panel.1")
# Update state
registry.update_state("my_process", ProcessState.RUNNING, pid=1234)
# Get snapshot for UI (no locks needed by caller)
snapshot = registry.get_snapshot()
# Remove requester
if registry.unregister("my_process", "panel.1"):
    # No more requesters, can stop process
    pass
```

10.1.1 Method: get

Get process info.

Returns a copy to prevent external mutation.

Arguments:

- name
/ *Condition*: required / *Type*: str /
The name of the process to retrieve.

Returns:

/ *Type*: Optional[ProcessInfo] /
A copy of the ProcessInfo, or None if not found.

10.1.2 Method: exists

Check if process is registered.

Arguments:

- name
/ *Condition*: required / *Type*: str /
The name of the process to check.

Returns:

/ *Type*: bool /
True if the process exists in the registry, False otherwise.

10.1.3 Method: get_all_names

Get all registered process names.

Returns:

/ *Type*: list[str] /
List of all registered process names.

10.1.4 Method: get_snapshot

Get immutable snapshot of all processes.

Returns frozen dataclass instances for UI path. No locks needed by caller - snapshot is immutable.

This replaces:

```
with shared_info_lock:
    proc_data = copy.deepcopy(proc_info)
```

Returns:

/ *Type*: tuple[ProcessSnapshot, ...] /
Tuple of immutable ProcessSnapshot objects.

10.1.5 Method: `get_requesters`

Get requester list for a process.

Arguments:

- `name`
/ *Condition*: required / *Type*: str /
The name of the process.

Returns:

/ *Type*: list[str] /
List of panel IDs that requested this process.

10.1.6 Method: `get_state`

Get process state.

Arguments:

- `name`
/ *Condition*: required / *Type*: str /
The name of the process.

Returns:

/ *Type*: Optional[ProcessState] /
The process state, or None if process not found.

10.1.7 Method: `register`

Register a process (or add requester if exists).

Arguments:

- `name`
/ *Condition*: required / *Type*: str /
The name of the process to register.
- `requester`
/ *Condition*: optional / *Type*: Optional[str] / *Default*: None /
Panel ID requesting this process.

Returns:

/ *Type*: bool /
True if newly registered, False if already existed.

10.1.8 Method: `unregister`

Remove requester from process.

Arguments:

- `name`
/ *Condition*: required / *Type*: str /
The name of the process.

- `requester`
/ *Condition*: optional / *Type*: Optional[str] / *Default*: None /
Panel ID to remove from requesters.

Returns:

/ *Type*: bool /
True if process has no more requesters (can be stopped).

10.1.9 Method: remove

Completely remove process from registry.

Arguments:

- `name`
/ *Condition*: required / *Type*: str /
The name of the process to remove.

Returns:

/ *Type*: bool /
True if process was removed, False if not found.

10.1.10 Method: update_state

Update process state.

Arguments:

- `name`
/ *Condition*: required / *Type*: str /
The name of the process to update.
- `state`
/ *Condition*: required / *Type*: ProcessState /
The new state to set.
- `pid`
/ *Condition*: optional / *Type*: Optional[int] / *Default*: None /
Process ID to set (if provided).
- `error`
/ *Condition*: optional / *Type*: Optional[str] / *Default*: None /
Error message (for FAILED state).

Returns:

/ *Type*: bool /
True if process found and updated.

10.1.11 Method: `update_pid`

Update process PID.

Arguments:

- `name`
/ *Condition*: required / *Type*: str /
The name of the process.
- `pid`
/ *Condition*: required / *Type*: int /
The new process ID.

Returns:

/ *Type*: bool /
True if process found and updated.

10.1.12 Method: `remove_requester_from_all`

Remove requester from all processes.

Used when a panel disconnects.

Arguments:

- `requester`
/ *Condition*: required / *Type*: str /
The panel ID to remove from all processes.

Returns:

/ *Type*: list[str] /
List of process names that now have no requesters (can be stopped).

10.1.13 Method: `get_active_processes`

Get names of processes in RUNNING state.

Returns:

/ *Type*: list[str] /
List of process names that are currently running.

10.1.14 Method: `mark_dead`

Mark processes as DEAD.

Called after health check detects dead processes.

Arguments:

- `names`
/ *Condition*: required / *Type*: list[str] /
List of process names to mark as dead.

10.1.15 Method: `clear_all`

Clear all processes (for shutdown).

Chapter 11

restart_state.py

Restart state machine - explicit FSM for restart coordination.

This module implements improvement 3.3:

- Define RestartState enum with explicit states
- RestartContext holds all restart-related data
- RestartStateMachine manages state transitions under single lock

Benefits: - Logic becomes easier to reason about

- Future features (timeout, partial restart, retry) become possible
- All restart state protected by single lock

11.1 Class: RestartState

Imported by:

```
from ProcessHub.core.restart_state import RestartState
```

RestartState: Explicit restart states.

Replaces implicit state spread across:

- self.is_notify flag
- self.killed_process list
- self.not_ready_panels list
- self.restart_notify_panels list
- threading.Event flags

11.2 Class: RestartContext

Imported by:

```
from ProcessHub.core.restart_state import RestartContext
```

RestartContext: All restart-related state in one place.

Replaces scattered state:

- self.killed_process -> killed_processes

- self.not_ready_panels -> pending_panels
- self.restart_notify_panels -> notified_panels
- self.is_notify -> state == NOTIFYING/AWAITING_ACK

New fields for future features:

- started_at: for timeout detection (was missing)
- retry_count: for max retry logic (was missing)
- error_message: for failure reporting (was missing)

11.2.1 Method: reset

Reset to idle state.

Clears all restart-related data except retry_count which accumulates across restart attempts.

11.2.2 Method: is_timed_out

Check if restart has exceeded timeout.

Returns:

/ *Type*: bool /

True if the restart has exceeded the timeout_seconds, False otherwise.

11.2.3 Method: can_retry

Check if retry is allowed.

Returns:

/ *Type*: bool /

True if retry_count is less than max_retries.

11.2.4 Method: elapsed_time

Get elapsed time since restart started.

Returns:

/ *Type*: Optional[float] /

Elapsed time in seconds, or None if restart hasn't started.

11.3 Class: RestartStateMachine

Imported by:

```
from ProcessHub.core.restart_state import RestartStateMachine
```

RestartStateMachine: State machine for restart coordination.

All transitions are explicit and guarded. Single lock protects all state transitions.

State diagram:

```

IDLE -> DETECTING -> NOTIFYING -> AWAITING_ACK -> RESTARTING -> DONE -> IDLE
-----|-----|
-----v-----v
-----> FAILED -----> IDLE (retry or give up)
```

```

Usage: fsm = RestartStateMachine()
        # Health check detected dead processes
        if fsm.detect_killed(["proc1"], ["panel1", "panel2"]):

            send_notifications()
            fsm.notifications_sent()

        # Panel acknowledged
        success, all_ready = fsm.panel_acknowledged("panel1")
        if all_ready and fsm.begin_restart():

            do_restart()
            fsm.restart_completed(success=True)
            fsm.reset_to_idle()

```

11.3.1 Method: state

Get current state (thread-safe).

Returns:

/ Type: RestartState /
The current restart state.

11.3.2 Method: killed_processes

Get killed processes (thread-safe copy).

Returns:

/ Type: list[str] /
Copy of the list of killed process names.

11.3.3 Method: pending_panels

Get pending panels (thread-safe copy).

Returns:

/ Type: Set[str] /
Copy of the set of panels that haven't acknowledged yet.

11.3.4 Method: get_snapshot

Get immutable snapshot of restart state.

Returns dict suitable for HubStateSnapshot.

Returns:

/ Type: dict /
Dictionary containing all restart state information.

11.3.5 Method: detect_killed

Transition: IDLE -> DETECTING -> NOTIFYING

Called when health check detects killed processes.

Arguments:

- `killed_processes`
/ *Condition*: required / *Type*: list[str] /
List of dead process names.
- `affected_panels`
/ *Condition*: required / *Type*: list[str] /
List of panel IDs that need notification.

Returns:

/ *Type*: bool /
True if transition successful, False if restart already in progress.

11.3.6 Method: notifications_sent

Transition: NOTIFYING -> AWAITING_ACK

Called after all restart notifications have been sent.

Returns:

/ *Type*: bool /
True if transition successful.

11.3.7 Method: panel_acknowledged

Record panel acknowledgment.

Arguments:

- `panel_id`
/ *Condition*: required / *Type*: str /
Panel ID that acknowledged.

Returns:

/ *Type*: tuple[bool, bool] /
Tuple of (success, all_acked): - success: True if acknowledgment was recorded - all_acked:
True if all panels have now acknowledged

11.3.8 Method: begin_restart

Transition: AWAITING_ACK -> RESTARTING

Called when all panels have acknowledged.

Returns:

/ *Type*: bool /
True if transition successful.

11.3.9 Method: restart_completed

Transition: RESTARTING -> DONE | FAILED

Arguments:

- `success`
/ *Condition*: required / *Type*: bool /
Whether restart succeeded.

- `error_message`
/ *Condition*: optional / *Type*: Optional[str] / *Default*: None /
Error details if failed.

Returns:

/ *Type*: bool /
True if transition successful.

11.3.10 Method: reset_to_idle

Transition: DONE | FAILED -> IDLE

Called after notifying clients of restart completion.

11.3.11 Method: check_timeout

Check for timeout and transition to FAILED if exceeded.

Should be called periodically during AWAITING_ACK or RESTARTING states.

Returns:

/ *Type*: bool /
True if timed out (state changed to FAILED).

11.3.12 Method: force_fail

Force transition to FAILED state.

Use for unrecoverable errors during restart.

Arguments:

- `error_message`
/ *Condition*: required / *Type*: str /
Error message describing the failure.

Returns:

/ *Type*: bool /
True if state was changed.

Chapter 12

`__init__.py`

Fleet orchestrator package - multi-hub coordination overlay.

Provides: - FleetOrchestrator: Central coordinator for multiple hubs - HubAgent: Sidecar alongside each ProcessHub-Server - FleetClient: External API for CI/CD and CLI - HubRegistry: Thread-safe hub tracking - FleetTopics: Message topic enum - Fleet models: Protocol messages and frozen snapshots

Optional: - FleetWebAPI: REST dashboard (requires FastAPI)

Chapter 13

fleet_client.py

Fleet client - external API for CI/CD pipelines and CLI tools.

Provides high-level methods to: - Query fleet and hub status - Start/stop processes on specific hubs - Reset hubs - Batch operations across multiple hubs - Event callbacks for hub online/offline events

Usage:

```
from ProcessHub.fleet import FleetClient
client = FleetClient(transport=fleet_transport, orchestrator=orchestrator)
# Check fleet health snapshot = client.get_fleet_status() assert snapshot.online_hubs >= 2
# Start processes on a specific hub client.start_processes("bench-1", ["ecu_simulator", "can_bridge"])
# Stop all processes on all hubs client.stop_all_processes()
```

13.1 Class: FleetClient

Imported by:

```
from ProcessHub.fleet.fleet_client import FleetClient
```

Fleet client for external systems.

Wraps FleetOrchestrator to provide a clean API for CI/CD, CLI, and scripts. Supports event callbacks for hub lifecycle events.

13.1.1 Method: get_fleet_status

Get current fleet-wide status.

Returns:

/ Type: FleetStateSnapshot /
Immutable snapshot of the entire fleet.

13.1.2 Method: get_hub_status

Get status of a specific hub.

Arguments:

- `hub_id`
/ Condition: required / Type: str /
The hub to query.

Returns:

/ Type: Optional[HubSnapshot] /
Hub snapshot if found, None otherwise.

13.1.3 Method: start_processes

Start processes on a specific hub.

Arguments:

- `hub_id`
/ Condition: required / Type: str /
Target hub identifier.
- `process_list`
/ Condition: required / Type: list[str] /
Process names to start.
- `panel_id`
/ Condition: optional / Type: str / Default: "fleet" /
Panel ID for process ownership.

Returns:

/ Type: str /
Command ID for result tracking.

13.1.4 Method: stop_processes

Stop processes on a specific hub.

Arguments:

- `hub_id`
/ Condition: required / Type: str /
Target hub identifier.
- `process_list`
/ Condition: required / Type: list[str] /
Process names to stop.
- `panel_id`
/ Condition: optional / Type: str / Default: "fleet" /
Panel ID for process ownership.
- `force`
/ Condition: optional / Type: bool / Default: False /
Force stop (kill) processes.

Returns:

/ Type: str /
Command ID for result tracking.

13.1.5 Method: reset_hub

Reset a specific hub (clear all state).

Arguments:

- `hub_id`
/ *Condition*: required / *Type*: str /
Target hub identifier.

Returns:

/ *Type*: str /
Command ID for result tracking.

13.1.6 Method: get_hub_detail

Request detailed status from a specific hub.

Arguments:

- `hub_id`
/ *Condition*: required / *Type*: str /
Target hub identifier.

Returns:

/ *Type*: str /
Command ID for result tracking.

13.1.7 Method: start_on_all_hubs

Start processes on all online hubs.

Arguments:

- `process_list`
/ *Condition*: required / *Type*: list[str] /
Process names to start on each hub.
- `panel_id`
/ *Condition*: optional / *Type*: str / *Default*: "fleet" /
Panel ID for process ownership.

Returns:

/ *Type*: dict[str, str] /
Mapping of `hub_id` to `command_id`.

13.1.8 Method: stop_all_processes

Stop all processes on all online hubs.

Arguments:

- `panel_id`
/ *Condition*: optional / *Type*: str / *Default*: "fleet" /
Panel ID for process ownership.

- `force`
/ *Condition*: optional / *Type*: bool / *Default*: False /
Force stop (kill) processes.

Returns:

/ *Type*: dict[str, str] /
Mapping of `hub_id` to `command_id`.

13.1.9 Method: reset_all_hubs

Reset all online hubs.

Returns:

/ *Type*: dict[str, str] /
Mapping of `hub_id` to `command_id`.

13.1.10 Method: on_hub_online

Register a callback for hub online events.

Arguments:

- `callback`
/ *Condition*: required / *Type*: Callable[[dict], None] /
Called with hub data when a hub comes online.

13.1.11 Method: on_hub_offline

Register a callback for hub offline events.

Arguments:

- `callback`
/ *Condition*: required / *Type*: Callable[[dict], None] /
Called with hub data when a hub goes offline.

Chapter 14

hub_agent.py

Hub agent - lightweight sidecar alongside each ProcessHubServer.

The HubAgent runs alongside a local ProcessHubServer and provides: - Periodic heartbeat (HubAnnounce) for liveness detection - Periodic status reports (HubStatusReport) with server state - Fleet command handler (start/stop/reset via local server APIs)

The agent uses only the server's public API - no core modifications needed.

14.1 Class: HubAgent

Imported by:

```
from ProcessHub.fleet.hub_agent import HubAgent
```

Hub agent sidecar for ProcessHubServer.

Sits alongside a local ProcessHubServer and bridges it to the fleet: - Announces hub to orchestrator via periodic heartbeats - Reports server status periodically - Handles fleet commands by delegating to local server

Usage:

```
from ProcessHub.fleet import HubAgent

agent = HubAgent( server=my_server, transport=fleet.transport, hub_id="bench-1", hub_name="HIL
    Bench 1",

) agent.start() # ... server runs ... agent.stop()
```

14.1.1 Method: hub_id

Get the hub ID. processhub-fleet-hub-agent-hubagent-hub-name -----

Get the hub name. processhub-fleet-hub-agent-hubagent-is-running -----

Check if the agent is running. processhub-fleet-hub-agent-hubagent-start -----

Start the hub agent.

Registers fleet command handler, sends initial announcement, and starts background heartbeat and status report loops.

14.1.2 Method: stop

Stop the hub agent.

Sends deregistration message and stops background loops.

Chapter 15

hub_registry.py

Thread-safe hub registry for fleet orchestrator.

Tracks registered hubs with health state transitions: online -> degraded -> offline

Provides immutable FleetStateSnapshot for concurrent consumers (REST API, FleetClient, orchestrator tick).

15.1 Class: _HubEntry

Imported by:

```
from ProcessHub.fleet.hub-registry import _HubEntry
```

Mutable internal hub tracking entry. processhub-fleet-hub-registry-hubregistry =====

Imported by:

```
from ProcessHub.fleet.hub-registry import HubRegistry
```

Thread-safe registry of fleet hubs.

Provides: - Hub registration and deregistration - Status and heartbeat updates - Health checking with timeout-based state transitions - Immutable snapshots for concurrent consumers

Health states: - online: heartbeat received within timeout/2 - degraded: heartbeat missed, within timeout - offline: heartbeat missed, exceeded timeout

15.1.1 Method: register

Register a hub in the fleet.

Arguments:

- `hub_id`
/ *Condition*: required / *Type*: str /
Unique identifier for the hub.
- `info`
/ *Condition*: required / *Type*: HubAnnounce /
Hub announcement message with hub details.

Returns:

/ *Type*: bool /
True if newly registered, False if already existed (updated instead).

15.1.2 Method: deregister

Remove a hub from the fleet.

Arguments:

- `hub_id`
/ *Condition*: required / *Type*: str /
The hub to remove.

Returns:

/ *Type*: bool /
True if hub was found and removed, False if not found.

15.1.3 Method: update_status

Update hub status from a HubStatusReport message.

Arguments:

- `hub_id`
/ *Condition*: required / *Type*: str /
The hub to update.
- `report`
/ *Condition*: required / *Type*: HubStatusReport /
Status report from the hub agent.

15.1.4 Method: update_heartbeat

Update the last-seen timestamp for a hub.

Called when any message is received from the hub.

Arguments:

- `hub_id`
/ *Condition*: required / *Type*: str /
The hub to update.

15.1.5 Method: get_hub

Get an immutable snapshot of a single hub.

Arguments:

- `hub_id`
/ *Condition*: required / *Type*: str /
The hub to look up.

Returns:

/ *Type*: Optional[HubSnapshot] /
Hub snapshot if found, None otherwise.

15.1.6 Method: get_all_hubs

Get immutable snapshots of all registered hubs.

Returns:

/ Type: list[HubSnapshot] /
List of hub snapshots.

15.1.7 Method: get_online_hubs

Get immutable snapshots of all online hubs.

Returns:

/ Type: list[HubSnapshot] /
List of hub snapshots with status 'online'.

15.1.8 Method: is_registered

Check if a hub is registered.

Arguments:

- `hub_id`
/ Condition: required / Type: str /
The hub to check.

Returns:

/ Type: bool /
True if hub is registered.

15.1.9 Method: check_health

Check for timed-out hubs and update health states.

Called periodically by the orchestrator `tick()`.

Health transitions: - `last_seen` within `timeout/2`: `online` - `last_seen` within `timeout`: `degraded` - `last_seen` exceeded `timeout`: `offline` (returned in list)

Arguments:

- `timeout_seconds`
/ Condition: required / Type: float /
Maximum time since last heartbeat before a hub is considered offline.

Returns:

/ Type: list[str] /
List of hub IDs that just transitioned to offline.

15.1.10 Method: get_snapshot

Get an immutable snapshot of the entire fleet state.

Thread-safe: acquires lock, copies state, releases lock. Consumers can read the snapshot without locks.

Returns:

/ Type: FleetStateSnapshot /
Immutable fleet state snapshot.

15.1.11 Method: hub_count

Get total number of registered hubs.

Chapter 16

models.py

Fleet protocol messages and immutable state snapshots.

Messages: - HubAnnounce: Hub agent heartbeat / registration - HubRegistered: Orchestrator acknowledgement - HubDeregistered: Hub removal notification - HubStatusReport: Periodic status from hub agent - FleetCommand: Command from orchestrator to hub agent - FleetCommandResult: Result from hub agent to orchestrator

Snapshots (frozen): - HubSnapshot: Immutable view of a single hub - FleetStateSnapshot: Immutable view of the entire fleet

16.1 Class: HubAnnounce

Imported by:

```
from ProcessHub.fleet.models import HubAnnounce
```

HubAnnounce: Hub agent heartbeat and registration message.

Sent periodically by each HubAgent to announce its presence. The orchestrator uses this for hub discovery and liveness detection.

16.2 Class: HubRegistered

Imported by:

```
from ProcessHub.fleet.models import HubRegistered
```

HubRegistered: Orchestrator -> HubAgent registration acknowledgement.

Sent by the orchestrator after successfully registering a hub.

16.3 Class: HubDeregistered

Imported by:

```
from ProcessHub.fleet.models import HubDeregistered
```

HubDeregistered: Hub removal notification.

Sent when a hub is deregistered (graceful shutdown or timeout).

16.4 Class: HubStatusReport

Imported by:

```
from ProcessHub.fleet.models import HubStatusReport
```

HubStatusReport: Periodic status from hub agent.

Sent by HubAgent with current server state from `server.core.get_state_snapshot()`.

16.5 Class: FleetCommand

Imported by:

```
from ProcessHub.fleet.models import FleetCommand
```

FleetCommand: Command from orchestrator/client to hub agent.

Supported actions: - `start_process`: `params = {"process_list": [...], "panel_id": "fleet"}` - `stop_process`: `params = {"process_list": [...], "panel_id": "fleet"}` - `reset`: `params = {}` - `get_status`: `params = {}`

16.6 Class: FleetCommandResult

Imported by:

```
from ProcessHub.fleet.models import FleetCommandResult
```

FleetCommandResult: Result from hub agent back to orchestrator.

Contains the `command_id` for correlation with the original FleetCommand.

16.7 Class: FleetStatusRequest

Imported by:

```
from ProcessHub.fleet.models import FleetStatusRequest
```

FleetStatusRequest: Request for fleet-wide status.

Sent by FleetClient to request a FleetStatusResponse from the orchestrator.

16.8 Class: FleetStatusResponse

Imported by:

```
from ProcessHub.fleet.models import FleetStatusResponse
```

FleetStatusResponse: Fleet-wide status from orchestrator.

Contains serialized fleet state for the requesting client.

16.9 Class: HubSnapshot

Imported by:

```
from ProcessHub.fleet.models import HubSnapshot
```

HubSnapshot: Immutable snapshot of a single hub's state.

Used by FleetWebAPI, FleetClient, and orchestrator for thread-safe fleet state queries without holding locks.

16.10 Class: FleetStateSnapshot

Imported by:

```
from ProcessHub.fleet.models import FleetStateSnapshot
```

FleetStateSnapshot: Immutable snapshot of the entire fleet.

Returned by FleetOrchestrator.get_fleet_snapshot() and HubRegistry.get_snapshot().

Chapter 17

orchestrator.py

Fleet orchestrator - central coordinator for multiple ProcessHub instances.

The FleetOrchestrator: - Discovers hubs via HubAnnounce messages - Tracks hub health via heartbeat timeouts - Routes commands from FleetClient to target hubs - Provides fleet-wide state snapshots - Broadcasts hub online/offline events

Usage:

```
from ProcessHub.fleet import FleetOrchestrator

orchestrator = FleetOrchestrator( transport=fleet_transport, health_timeout=30.0,

) orchestrator.start()
# Main loop while True: orchestrator.tick() time.sleep(5)
orchestrator.stop()
```

17.1 Class: FleetOrchestrator

Imported by:

```
from ProcessHub.fleet.orchestrator import FleetOrchestrator
```

Fleet orchestrator - central coordinator.

Responsibilities: - Hub discovery and registration - Health monitoring via heartbeat timeouts - Command routing to target hubs - Fleet-wide state snapshots

The orchestrator is passive: it reacts to hub messages and provides APIs for FleetClient and FleetWebAPI to use.

17.1.1 Method: registry

Get the hub registry. processhub-fleet-orchestrator-fleetorchestrator-is-running -----

Check if the orchestrator is running. processhub-fleet-orchestrator-fleetorchestrator-start -----

Start the fleet orchestrator.

Registers message handlers and starts the transport.

17.1.2 Method: stop

Stop the fleet orchestrator.

17.1.3 Method: tick

Periodic health check and maintenance.

Should be called periodically (e.g., every `tick_interval` seconds). Checks hub health, marks offline hubs, and broadcasts `HUB_OFFLINE` events.

17.1.4 Method: send_command

Send a command to a specific hub.

Arguments:

- `hub_id`
/ *Condition*: required / *Type*: str /
Target hub identifier.
- `action`
/ *Condition*: required / *Type*: str /
Command action (`start_process`, `stop_process`, `reset`, `get_status`).
- `params`
/ *Condition*: optional / *Type*: Optional[dict] / *Default*: None /
Action parameters.

Returns:

/ *Type*: str /
Command ID for result correlation.

Raises:

`ValueError`: If hub is unknown or offline.

17.1.5 Method: get_fleet_snapshot

Get an immutable snapshot of the fleet state.

Returns:

/ *Type*: `FleetStateSnapshot` /
Current fleet state.

17.1.6 Method: get_hub_snapshot

Get an immutable snapshot of a single hub.

Arguments:

- `hub_id`
/ *Condition*: required / *Type*: str /
The hub to look up.

Returns:

/ *Type*: Optional[`HubSnapshot`] /
Hub snapshot if found, `None` otherwise.

Chapter 18

topics.py

Fleet message topics for EventBus routing.

All fleet topics use `FLEET_` prefix to separate from local hub topics (`PROCESS_*`, `REGISTER_*`, `SERVER_*`).

When used with `EventBusTransport`, the full routing key becomes: `fleet.FLEET_HUB_ANNOUNCE`, `fleet.FLEET_COMMAND`, etc.

18.1 Class: FleetTopics

Imported by:

```
from ProcessHub.fleet.topics import FleetTopics
```

FleetTopics: Message topics for fleet protocol.

Used with `transport.register_handler()` and `transport.send()`.

Chapter 19

web_api.py

Optional REST API and web dashboard for fleet orchestrator.

Provides: - REST API endpoints for fleet status, hub details, and command routing - HTML dashboard for fleet monitoring - Swagger/OpenAPI documentation at /docs

Requires FastAPI and uvicorn: pip install fastapi uvicorn

19.1 Class: CommandRequest

Imported by:

```
from ProcessHub.fleet.web_api import CommandRequest
```

Request body for fleet command endpoint. processhub-fleet-web-api-commandresponse =====

Imported by:

```
from ProcessHub.fleet.web_api import CommandResponse
```

Response body for fleet command endpoint. processhub-fleet-web-api-processactionrequest =====

Imported by:

```
from ProcessHub.fleet.web_api import ProcessActionRequest
```

Request body for hub process start/stop endpoints. processhub-fleet-web-api-fleetwebapi =====

Imported by:

```
from ProcessHub.fleet.web_api import FleetWebAPI
```

REST API and web dashboard for fleet orchestrator.

Usage:

```
from ProcessHub.fleet.web_api import FleetWebAPI
web_api = FleetWebAPI(orchestrator=orchestrator, host="0.0.0.0", port=2510) web_api.start() # ...
web_api.stop()
```

19.1.1 Method: start

Start the web server in a background thread. processhub-fleet-web-api-fleetwebapi-stop -----

Stop the web server.

Chapter 20

`__init__.py`

Logging server integration for Process Hub.

This module provides optional integration with centralized logging servers (Fluentbit, Telegraf, or custom implementations).

Usage:

```
from ProcessHub.logging import LoggingIntegration, LoggingServerType
# Initialize logging
logging = LoggingIntegration(
    server_type=LoggingServerType.FLUENTBIT,
    paths={
        "src_dir": "/path/to/fluentbit",
        "hot_dir": "/path/to/logs/fluentbit",
        "influx_dir": "/path/to/influxdb",
        "ctrl_dir": "/path/to/control",
    }
)
logging.start()
# Get config for a process
log_config = logging.get_process_logging_config("my_process")
# Cleanup
logging.stop()
```

Chapter 21

integration.py

Logging server integration.

This module provides integration with centralized logging servers, mirroring the functionality of `ProcessControl.init_logging_server` from the original ta-framework implementation.

The design is pluggable:

- Define a protocol for logging servers
- Support Fluentbit, Telegraf, or custom implementations
- Integrate with `ProcessHubServer` via composition

21.1 Class: `LoggingServerType`

Imported by:

```
from ProcessHub.logging.integration import LoggingServerType
```

Supported logging server types.

21.2 Class: `LoggingConfig`

Imported by:

```
from ProcessHub.logging.integration import LoggingConfig
```

Logging configuration for a process.

This is passed to the process executor to add logging arguments.

21.2.1 Method: `to_command_args`

Convert to command line arguments.

Matches the original format:

```
--variable TAF_PATH:{taf_path}  
--variable central_logging_conf:{config_dir}  
--variable central_logging_host:{host}  
--variable central_logging_port:{port}
```

21.2.2 Method: `to_dict`

Convert to dict for process config.

21.3 Class: LoggingServerProtocol

Imported by:

```
from ProcessHub.logging.integration import LoggingServerProtocol
```

Protocol for logging server implementations.

Any logging server (Fluentbit, Telegraf, custom) must implement these methods to work with LoggingIntegration.

21.3.1 Method: hot_cfg_dir

Get the hot configuration directory.

21.3.2 Method: hot_lock_cfg_path

Get the hot lock configuration file path.

21.3.3 Method: allocate

Allocate logging ports.

Args:

count: Number of ports to allocate

Returns:

Tuple of (host, list of ports)

21.3.4 Method: deallocate

Deallocate/cleanup the logging server.

21.4 Class: LoggingServerBase

Imported by:

```
from ProcessHub.logging.integration import LoggingServerBase
```

Abstract base class for logging servers.

Provides common functionality and defines the interface. Subclasses must implement allocate() and deallocate().

21.4.1 Method: hot_cfg_dir

21.4.2 Method: hot_lock_cfg_path

21.4.3 Method: allocate

Allocate logging ports.

21.4.4 Method: deallocate

Deallocate/cleanup.

21.5 Class: MockLoggingServer

Imported by:

```
from ProcessHub.logging.integration import MockLoggingServer
```

Mock logging server for testing.

Simulates port allocation without actually starting any services.

21.5.1 Method: allocate

Allocate mock ports.

21.5.2 Method: deallocate

Deallocate mock resources.

21.6 Class: LoggingIntegration

Imported by:

```
from ProcessHub.logging.integration import LoggingIntegration
```

Logging server integration manager.

This class manages the lifecycle of a logging server and provides methods to get logging configuration for processes.

Mirrors the functionality of `ProcessControl.init_logging_server()` from the original implementation.

Usage:

```
# Initialize
logging = LoggingIntegration(
    server_type=LoggingServerType.FLUENTBIT,
    paths={"src_dir": ..., "hot_dir": ..., ...}
)
logging.start()
# Get config for a process
config = logging.get_process_logging_config("my_process")
# Use in process start
if config.enabled:
    extra_args = config.to_command_args(
        taf_path="/path/to/taf")
# Cleanup
logging.stop()
```

With custom server factory:

```
def my_factory(uuid, paths):
    return MyCustomLoggingServer(
        uuid=uuid, **paths)

logging = LoggingIntegration(
    server_type=LoggingServerType.CUSTOM,
    server_factory=my_factory,
)
```

21.6.1 Method: server

Get the logging server instance.

21.6.2 Method: config_dir

Get the logging configuration directory.

21.6.3 Method: is_started

Check if logging server is started.

21.6.4 Method: start

Start the logging server.

Equivalent to `ProcessControl.init_logging_server()`.

21.6.5 Method: stop

Stop the logging server and clean up.

Equivalent to `ProcessControl.__del__()` cleanup.

21.6.6 Method: get_process_logging_config

Get logging configuration for a process.

Args:

`process_name`: Name of the process

`process_config`: Process configuration dict (to check `central_log` flag)

Returns:

LoggingConfig with allocated port if `central_log` is enabled

21.6.7 Method: enhance_process_config

Enhance process config with logging settings.

Args:

`process_name`: Name of the process

`process_config`: Original process configuration

Returns:

Enhanced config with logging settings added

21.7 Class: LoggingAwareExecutor

Imported by:

```
from ProcessHub.logging.integration import LoggingAwareExecutor
```

Executor wrapper that adds logging configuration to process starts.

Wraps an existing executor and enhances process configs with logging.

21.7.1 Method: start

Start process with logging config.

21.7.2 Method: stop

Stop process.

21.7.3 Method: is_running

Check if process is running.

Chapter 22

`__init__.py`

Process execution layer.

Provides abstract process executor interface and implementations:

- `ProcessExecutor`: Abstract interface
- `SimpleExecutor`: Basic cross-platform executor
- `MockExecutor`: For testing
- `ProcessManager`: High-level process lifecycle management

Platform-specific executors:

- `WindowsExecutor`: Windows-specific (`cmd.exe`, `PowerShell`)
- `UnixExecutor`: Unix/Linux/macOS (POSIX signals)

Chapter 23

executor.py

Process executor interface and implementations.

This module provides:

- Abstract ProcessExecutor interface
- SimpleExecutor for basic subprocess management
- Factory function for platform-appropriate executor

23.1 Function: create_executor

Factory function to create an executor.

Args:

executor_type: "simple", "mock", or "windows"

Returns:

ProcessExecutor implementation

23.2 Class: ProcessExecutor

Imported by:

```
from ProcessHub.process.executor import ProcessExecutor
```

Abstract process executor interface.

Implementations handle platform-specific process management.

23.2.1 Method: start

Start a process.

Args:

name: Process name/identifier

config: Process configuration dict with keys:

- script: Path to script/executable
- args: List of arguments
- wait_time: Time to wait after start (seconds)
- process_name: Actual process name to look for

- env: Additional environment variables

Returns:

Tuple of (success, message, pid)

23.2.2 Method: stop

Stop a process.

Args: name: Process name/identifier force: Force kill if graceful stop fails

Returns: Tuple of (success, message)

23.2.3 Method: is_running

Check if a process is running.

Args:

name: Process name/identifier

Returns:

True if process is running

23.2.4 Method: get_pid

Get process ID.

Args:

name: Process name/identifier

Returns:

PID or None if not running

23.3 Class: SimpleExecutor

Imported by:

```
from ProcessHub.process.executor import SimpleExecutor
```

Simple process executor using subprocess.

This executor:

- Tracks processes by name in a dict
- Uses psutil for process management (if available)
- Falls back to subprocess for basic operations

Suitable for testing and simple use cases.

For production Windows use, consider WindowsExecutor.

23.3.1 Method: start

Start a process.

23.3.2 Method: stop

Stop a process.

23.3.3 Method: is_running

Check if process is running.

23.3.4 Method: get_pid

Get process PID.

23.3.5 Method: update_process

Update tracked process (e.g., after finding external process).

Args:

name: Process name

process: Process object (psutil.Process or Popen)

pid: Process ID

23.4 Class: MockExecutor

Imported by:

```
from ProcessHub.process.executor import MockExecutor
```

Mock executor for testing.

Simulates process start/stop without actually running processes.

23.4.1 Method: start

Simulate starting a process.

23.4.2 Method: stop

Simulate stopping a process.

23.4.3 Method: is_running

Check if mock process is running.

23.4.4 Method: get_pid

Get mock PID.

23.4.5 Method: kill

Simulate a process being killed externally.

Chapter 24

manager.py

Process Manager - high-level process lifecycle management.

This module provides a unified interface for managing processes, combining the functionality of:

- Process execution (via Executor)
- Process state tracking (via ProcessRegistry from core)
- Requester management (reference counting)

This is the normalized version of the original ProcessManager, with explicit process states and cleaner separation of concerns.

24.1 Class: ProcessManager

Imported by:

```
from ProcessHub.process.manager import ProcessManager
```

High-level process manager.

Combines process execution with state tracking:

- Manages process lifecycle (register, start, stop)
- Tracks requesters (reference counting)
- Provides health checks
- Thread-safe operations

This is a normalized version of the original ProcessManager:

- Uses ProcessState enum instead of mixed "None"/True/False
- Uses ProcessRegistry for thread-safe state management
- Separates execution logic into Executor

Usage:

```
manager = ProcessManager()
# Register and start
manager.register("my_process", requester="panel_1", config={...})
success, msg = manager.start("my_process")
# Check status
if manager.is_running("my_process"):
```

```

    print("Running!")

    # Health check
    dead = manager.health_check()
    # Stop (only if no other requesters)
    manager.unregister("my_process", requester="panel_1")
    manager.stop("my_process") # Actually stops if no requesters

```

24.1.1 Method: executor

Get the executor.

24.1.2 Method: registry

Get the registry.

24.1.3 Method: register

Register a process.

Arguments:

- `name`
/ Condition: required / Type: str /
 Process name.
- `requester`
/ Condition: optional / Type: str / Default: None /
 Panel/client ID requesting this process.
- `config`
/ Condition: optional / Type: dict / Default: None /
 Process configuration (script, args, etc.).

Returns:

/ Type: bool /

True if newly registered, False if already existed.

24.1.4 Method: unregister

Unregister a requester from a process.

Arguments:

- `name`
/ Condition: required / Type: str /
 Process name.
- `requester`
/ Condition: optional / Type: str / Default: None /
 Panel/client ID to remove.

Returns:

/ Type: bool /

True if process has no more requesters (can be stopped).

24.1.5 Method: start

Start a process.

Arguments:

- name
/ *Condition*: required / *Type*: str /
Process name (must be registered first).

Returns:

/ *Type*: tuple[bool, str] /
Tuple of (success, message).

24.1.6 Method: stop

Stop a process.

Arguments:

- name
/ *Condition*: required / *Type*: str /
Process name.
- force
/ *Condition*: optional / *Type*: bool / *Default*: False /
Force stop even if requesters exist.

Returns:

/ *Type*: bool /
True if stopped successfully.

24.1.7 Method: force_stop_all

Force stop all processes.

24.1.8 Method: is_running

Check if a process is running.

24.1.9 Method: get_state

Get process state.

24.1.10 Method: get_info

Get process info.

24.1.11 Method: get_all_names

Get all registered process names.

24.1.12 Method: get_requesters

Get requesters for a process.

24.1.13 Method: `health_check`

Check health of all running processes.

Returns:

List of process names that are dead (were running, now not)

24.1.14 Method: `remove_requester_from_all`

Remove requester from all processes.

Arguments:

- `requester`
/ *Condition*: required / *Type*: str /
Requester to remove.

Returns:

/ *Type*: list[str] /

List of process names that now have no requesters.

24.1.15 Method: `get_shared_info`

Get shared info dict (backward compatibility).

Returns dict in format compatible with original ProcessManager:

```
{ "process_name": {  
    "pid": 1234,  
    "requester": ["panel_1", "panel_2"],  
    "status": True  
}
```

24.1.16 Method: `update_process`

Update process object in executor (backward compatibility).

Used when external process lookup finds a running process.

Chapter 25

`__init__.py`

Platform-specific process execution.

Provides platform-specific executor implementations:

- `PlatformExecutor`: Abstract base class
- `WindowsExecutor`: Windows-specific (`cmd.exe`, `PowerShell`)
- `UnixExecutor`: Unix/Linux/macOS (POSIX signals)

25.1 Function: `get_platform_executor`

Get the appropriate executor for the current platform.

Returns: Platform-specific executor instance

Chapter 26

base.py

Base platform executor interface.

This module defines the abstract base class for platform-specific process executors. Each platform (Windows, Unix) implements this interface with appropriate system calls.

26.1 Class: PlatformExecutor

Imported by:

```
from ProcessHub.process.platform.base import PlatformExecutor
```

Abstract base class for platform-specific process executors.

Subclasses implement platform-specific process management:

- WindowsExecutor: Uses cmd.exe, PowerShell, Win32 APIs
- UnixExecutor: Uses fork/exec, signals, /proc filesystem

Configuration dict keys (common):

- script: Path to script/executable
- args: List of arguments
- wait_time: Time to wait after start (seconds)
- process_name: Actual process name (for lookup)
- env: Additional environment variables
- working_dir: Working directory for process
- central_log: Whether to use central logging

26.1.1 Method: start

Start a process.

Args:

name: Process name/identifier
config: Process configuration

Returns:

Tuple of (success, message, pid)

26.1.2 Method: stop

Stop a process.

Args:

name: Process name/identifier
force: Force kill if graceful stop fails

Returns:

True if stopped successfully

26.1.3 Method: is_running

Check if a process is running.

Args: name: Process name/identifier

Returns: True if running

26.1.4 Method: get_pid

Get process ID.

Args:

name: Process name/identifier

Returns:

PID or None if not running

26.1.5 Method: update_process

Update tracked process.

Used when external lookup finds a running process.

Args:

name: Process name
process: Process object
pid: Process ID

26.1.6 Method: cleanup

Clean up tracking for a process.

Args:

name: Process name

26.1.7 Method: cleanup_all

Clean up all tracked processes.

Chapter 27

unix.py

Unix-specific process executor.

Uses POSIX APIs for process management:

- subprocess.Popen for starting processes
- os.kill / signal for termination
- psutil (if available) for process tracking

27.1 Class: UnixExecutor

Imported by:

```
from ProcessHub.process.platform.unix import UnixExecutor
```

Unix/Linux/macOS process executor.

Uses POSIX APIs:

- subprocess.Popen for starting
- SIGTERM/SIGKILL for stopping
- psutil for process tracking (if available)

Configuration:

- script: Path to script/executable
- args: List of arguments
- wait_time: Time to wait after start (default: 2.0)
- process_name: Process name for lookup
- env: Additional environment variables
- working_dir: Working directory
- daemon: Run as daemon (default: False)
- shell: Use shell execution (default: False)

27.1.1 Method: start

Start a process using subprocess.

27.1.2 Method: stop

Stop a process using signals.

27.1.3 Method: is_running

Check if process is running.

27.1.4 Method: find_process

Find a process by name.

Args:

name: Process name (used in command line)

process_name: Actual process name (e.g., "python")

Returns:

psutil.Process if found, None otherwise

Chapter 28

windows.py

Windows-specific process executor.

Uses Windows APIs for process management:

- cmd.exe with 'start' for launching minimized windows
- PowerShell for process termination
- psutil for process tracking

28.1 Class: WindowsExecutor

Imported by:

```
from ProcessHub.process.platform.windows import WindowsExecutor
```

Windows process executor using cmd.exe and PowerShell.

This executor matches the original ta-framework behavior:

- Uses 'start /min' to launch processes in minimized windows
- Uses PowerShell terminate script for stopping processes
- Uses psutil for process tracking and health checks

Configuration dict keys:

- script: Path to script/executable
- args: List of arguments
- wait_time: Time to wait after start (default: 2.0)
- process_name: Actual process name to search for (for psutil)
- central_log: Whether to use central logging
- python_exe: Python executable path (default: uses system python)

28.1.1 Method: start

Start a process using Windows cmd.exe start command.

28.1.2 Method: stop

Stop a process using PowerShell terminate script or psutil.

28.1.3 Method: is_running

Check if process is running using psutil.

28.1.4 Method: get_pid

Get process PID.

28.1.5 Method: update_process

Update tracked process.

Used when a process is found externally (e.g., already running).

Chapter 29

`__init__.py`

Runtime components - server, client, and transport wiring.

This module provides the runtime components that wire together the core logic with transport and UI layers.

Chapter 30

client.py

ProcessHubClient - client for connecting to ProcessHubServer.

This module provides a client API for:

- Registering/unregistering connections
- Starting/stopping processes
- Handling restart notifications

30.1 Class: ProcessHubClient

Imported by:

```
from ProcessHub.runtime.client import ProcessHubClient
```

Process Hub Client for connecting to ProcessHubServer.

Usage:

```
from ProcessHub.runtime.client import ProcessHubClient
from ProcessHub.transport.zmq_transport import ZmqTransport
transport = ZmqTransport(start_broker=False)
client = ProcessHubClient(
    transport=transport,
    panel_id="my_panel",
)
# Connect handlers
client.controller.connect("process_started", on_started)
client.controller.connect("process_killed", on_killed)
# Start client
client.start()
# Register with server
client.register_connection(session_id="my_session")
# Request processes
client.request_process_start(["system_manager", "edge_compute"])
# Handle restart (when on_killed is called)
client.notify_restart_ready()
# Cleanup
client.unregister_connection()
client.stop()
```

30.1.1 Method: start

Start the client (transport only).

30.1.2 Method: stop

Stop the client.

30.1.3 Method: panel_id

Get the panel ID.

30.1.4 Method: is_registered

Check if client is registered with server.

30.1.5 Method: is_running

Check if client is running.

30.1.6 Method: register_connection

Register connection with the server.

Args:

 session_id: Session identifier (groups related panels)
 timeout: Request timeout (uses default if None)

30.1.7 Method: wait_for_registration

Wait for registration to complete.

Call this after register_connection() to block until the server responds.

Args:

 timeout: Maximum time to wait (uses default_timeout if None)

Returns:

 True if registered successfully, False if timed out or failed.

Raises:

 RuntimeError: If registration failed with an error from server.

30.1.8 Method: unregister_connection

Unregister connection from the server.

Args:

 timeout: Request timeout

30.1.9 Method: request_connection_info

Request connection info from server.

Args:

 timeout: Request timeout

30.1.10 Method: request_process_start

Request to start process(es).

Args:

process_list: Process name(s) to start

timeout_per_process: Timeout per process (total = N * timeout)

30.1.11 Method: request_process_stop

Request to stop process(es).

Args:

process_list: Process name(s) to stop

force: Force stop even if other panels need the process

timeout_per_process: Timeout per process

30.1.12 Method: notify_restart_ready

Notify server that this panel is ready for restart.

Call this after receiving on_process_killed callback and completing any necessary cleanup.

Args:

timeout: Timeout waiting for restart completion

Chapter 31

client_controller.py

ClientController - callback handler registry for client events.

This is a simple dispatcher that maps action names to user-provided handlers. It decouples message arrival from application logic.

31.1 Class: ClientEvent

Imported by:

```
from ProcessHub.runtime.client_controller import ClientEvent
```

Enumeration of client events for type-safe event handling.

Use these enum values instead of strings to avoid typos and get IDE autocomplete.

Usage:

```
from ProcessHub.runtime import ClientEvent

# Register with enum (recommended)
client.controller.connect(ClientEvent.PROCESS_STARTED, on_started)

# Or use string (backward compatible)
client.controller.connect("process_started", on_started)
```

31.2 Class: ClientController

Imported by:

```
from ProcessHub.runtime.client_controller import ClientController
```

Client controller for handling process hub events.

Register callbacks for various events, and they will be called when the corresponding server response arrives.

Usage:

```
controller = ClientController()
def on_started(data):
    print(f"Processes started: {data}")

controller.connect("process_started", on_started)
# Later, when response arrives:
controller.on_process_started(response_data) # Calls on_started
```

31.2.1 Method: connect

Connect an action to a handler.

Arguments:

- `action`
/ *Condition*: required / *Type*: Union[str, ClientEvent] /
Event name as string or ClientEvent enum value.
- `handler`
/ *Condition*: required / *Type*: Callable /
Callable to invoke when action occurs.

Raises:

- `ValueError`: If action is unknown.
- `TypeError`: If handler is not callable.

31.2.2 Method: disconnect

Disconnect a handler from an action.

Arguments:

- `action`
/ *Condition*: required / *Type*: Union[str, ClientEvent] /
Event name as string or ClientEvent enum value.

31.2.3 Method: on_connection_registered

Called when connection registration response received.

Arguments:

- `data`
/ *Condition*: required / *Type*: dict /
Registration response data.

Returns:

/ *Type*: Any /

Handler return value or None.

31.2.4 Method: on_connection_unregistered

Called when connection unregistration response received.

Arguments:

- `data`
/ *Condition*: required / *Type*: dict /
Unregistration response data.

Returns:

/ *Type*: Any /

Handler return value or None.

31.2.5 Method: on_connection_info

Called when connection info response received.

Arguments:

- `connection_info`
/ *Condition*: required / *Type*: dict /
Connection information data.

Returns:

/ *Type*: Any /

Handler return value or None.

31.2.6 Method: on_process_started

Called when process start response received.

Arguments:

- `data`
/ *Condition*: required / *Type*: dict /
Response data with keys: `panel_id` (str), `success` (bool), `failed_processes` (list), `error_message` (str).

Returns:

/ *Type*: Any /

Handler return value or None.

31.2.7 Method: on_process_stopped

Called when process stop response received.

Arguments:

- `data`
/ *Condition*: required / *Type*: dict /
Response data with keys: `panel_id` (str), `success` (bool), `message` (str).

Returns:

/ *Type*: Any /

Handler return value or None.

31.2.8 Method: on_process_killed

Called when process restart notification received.

This indicates processes have died and restart is needed.

Arguments:

- `killed_processes`
/ *Condition*: required / *Type*: list[str] /
List of dead process names.

Returns:

/ *Type*: Any /

Handler return value or None.

31.2.9 Method: on_process_restarted

Called when process restart completed.

Arguments:

- data
/ *Condition*: required / *Type*: dict /
Response data with keys: panel_id (str), success (bool), failed_processes (list).

Returns:

/ *Type*: Any /

Handler return value or None.

31.2.10 Method: on_restart_failed

Called when restart failed (e.g., timeout, error).

Arguments:

- data
/ *Condition*: required / *Type*: dict /
Response data with keys: panel_id (str), success (False), failed_processes (list).

Returns:

/ *Type*: Any /

Handler return value or None.

31.2.11 Method: on_server_timeout

Called when server response times out.

Arguments:

- message
/ *Condition*: required / *Type*: str /
Timeout error message.

Returns:

/ *Type*: Any /

Handler return value or None.

31.2.12 Method: on_server_shutdown

Called when server shutdown or reset notification received.

This indicates the server is shutting down or resetting. The client should handle this by cleaning up state and potentially reconnecting.

Arguments:

- data
/ *Condition*: required / *Type*: dict /
Notification data with keys: panel_id (str), reason (str: "shutdown" or "reset"), message (str).

Returns:

/ *Type*: Any /

Handler return value or None.

Chapter 32

server.py

ProcessHubServer - thin runtime wrapper around ProcessHubCore.

This module implements improvement 3.1:

- ZmqHubServer = small adapter: recv ZMQ -> call core.handle_message -> send ZMQ
- All business logic is in ProcessHubCore
- Server only handles: transport wiring, main loop, lifecycle

Benefits:

- Clean separation of concerns
- Core can be tested without transport
- Easy to swap transports (ZMQ, HTTP, in-memory)

32.1 Class: Topics

Imported by:

```
from ProcessHub.runtime.server import Topics
```

Message topics for process hub protocol.

These match the original ProcessEventTopic values for compatibility.

32.2 Class: ProcessExecutor

Imported by:

```
from ProcessHub.runtime.server import ProcessExecutor
```

Protocol for process executor implementations.

32.2.1 Method: start

Start a process. Returns (success, message, pid).

32.2.2 Method: stop

Stop a process.

32.2.3 Method: `is_running`

Check if process is running.

32.3 Class: `ProcessHubServer`

Imported by:

```
from ProcessHub.runtime.server import ProcessHubServer
```

Process Hub Server - thin runtime wrapper.

Responsibilities:

- Wire transport handlers to core message handlers
- Run main loop (tick + UI refresh)
- Handle shutdown signals

All business logic is in `ProcessHubCore`.

Usage:

```
from ProcessHub.runtime.server import ProcessHubServer
from ProcessHub.transport.zmq_transport import ZmqTransport
from ProcessHub.ui.console_view import ConsoleView
transport = ZmqTransport(start_broker=True)
view = ConsoleView()
server = ProcessHubServer(
    transport=transport,
    view=view,
    process_config=my_config,
)
server.run()
```

32.3.1 Method: `start`

Start the server (transport only, no main loop).

32.3.2 Method: `stop`

Stop the server.

Sends shutdown notifications to all connected clients before stopping.

32.3.3 Method: `run`

Run the main loop (blocking).

This starts the server, runs the tick/render loop, and handles shutdown signals.

32.3.4 Method: reset

Reset the hub to a fresh state.

Stops all processes, clears all connections, and resets the restart state machine. The server continues running after reset. Sends reset notifications to all connected clients before clearing their connections.

Returns:

/ Type: tuple[bool, str] /

A tuple of (success, message) indicating result of reset operation.

32.3.5 Method: register_admin_process

Register a process started by admin (not via client request).

This allows processes started via the admin API to appear in the dashboard.

Arguments:

- name

/ Condition: required / Type: str /

The process name.

- pid

/ Condition: required / Type: int /

The process ID.

Returns:

/ Type: bool /

True if registered successfully.

32.3.6 Method: unregister_admin_process

Unregister an admin-started process.

Called when an admin-started process is stopped.

Arguments:

- name

/ Condition: required / Type: str /

The process name.

Returns:

/ Type: bool /

True if unregistered successfully.

32.3.7 Method: core

Get the core instance (for testing/introspection).

32.3.8 Method: configured_process_names

Get the list of configured process names.

Returns all process names from the process_config dict, regardless of whether they are currently running.

32.3.9 Method: `process_config`

Get the full process configuration dict.

Returns the mapping of process name to config dict, containing fields like script, args, env, wait_time, description, etc.

32.3.10 Method: `is_running`

Check if server is running.

Chapter 33

`__init__.py`

Transport layer - pluggable message transports.

Provides abstract transport interface and implementations:

- `TransportBase`: Abstract interface
- `InMemoryTransport`: For testing
- `ZmqTransport`: For production (requires `pyzmq`)
- `EventBusTransport`: For RabbitMQ-based messaging (requires `EventBusClient`)

Chapter 34

base.py

Abstract transport interface.

This module implements part of improvement 3.1:

- Abstract interface for message transport
- Allows ProcessHubCore to work with different transports

Benefits:

- ZMQ for production
- In-memory for testing
- HTTP/gRPC for future needs

34.1 Class: TransportBase

Imported by:

```
from ProcessHub.transport.base import TransportBase
```

Abstract transport interface.

Implementations must provide:

- start/stop lifecycle methods
- register_handler for receiving messages
- send/send_async for sending messages

Example implementation:

```
class MyTransport(TransportBase):  
    def start(self):  
        self._client = create_client()  
        self._client.connect()  
    def stop(self):  
        self._client.disconnect()  
    def register_handler(self, topic, handler):  
        self._client.subscribe(topic, handler)  
    def send(self, topic, message, target=None):  
        self._client.publish(topic, message)
```

34.1.1 Method: `start`

Start the transport.

Should initialize connections, start background threads, etc. Must be called before `send()` or `register_handler()`.

34.1.2 Method: `stop`

Stop the transport.

Should close connections, stop background threads, cleanup resources. Should be idempotent (safe to call multiple times).

34.1.3 Method: `register_handler`

Register message handler for topic.

Args:

topic: Topic name (string or Enum)

handler: Callable that receives deserialized message data

34.1.4 Method: `send`

Send message to topic (blocking).

Args:

topic: Topic name

message: Message data (will be serialized)

target: Optional target panel ID (for directed messages)

34.1.5 Method: `send_async`

Send message asynchronously (non-blocking).

Same as `send()` but returns immediately.

Chapter 35

eventbus_transport.py

EventBus transport implementation using RabbitMQ.

This module provides a RabbitMQ-based transport for the process hub, using the EventBusClient library for message passing.

Requires: EventBusClient (<https://github.com/test-fullautomation/python-rabbitmq-messagebus>)

Usage:

```
from ProcessHub.transport.eventbus_transport import EventBusTransport

# With config file
transport = EventBusTransport(config_path="config.jsonp")
transport.register_handler("TOPIC", my_handler)
transport.start()

# With inline config
transport = EventBusTransport(
    host="localhost",
    port=5672,
    exchange_name="process_hub",
)
transport.start()
transport.send("TOPIC", {"data": "value"})
```

35.1 Class: EventBusConfig

Imported by:

```
from ProcessHub.transport.eventbus_transport import EventBusConfig
```

Configuration for EventBus transport.

Usage:

```
# Minimal config
config = EventBusConfig(host="localhost")

# Full config
config = EventBusConfig(
    host="rabbitmq.example.com",
    port=5672,
    exchange_name="process_hub",
    exchange_handler="TopicExchangeHandler",
    serializer="JsonSerializer",
    auto_reconnect=True,
)
```

Attributes:

- `host`
/ *Type*: str / *Default*: "localhost" /
RabbitMQ server host.
- `port`
/ *Type*: int / *Default*: 5672 /
RabbitMQ server port.
- `exchange_name`
/ *Type*: str / *Default*: "process_hub" /
Exchange name for routing.
- `exchange_handler`
/ *Type*: str / *Default*: "TopicExchangeHandler" /
Exchange type handler class name.
- `serializer`
/ *Type*: str / *Default*: "PickleSerializer" /
Message serializer class name.
- `auto_reconnect`
/ *Type*: bool / *Default*: True /
Enable automatic reconnection on connection loss.
- `qos_prefetch`
/ *Type*: int / *Default*: 10 /
QoS prefetch count for message buffering.
- `routing_key_prefix`
/ *Type*: str / *Default*: "processhub" /
Prefix for all routing keys.

35.1.1 Method: to_dict

Convert to EventBusClient config format.

Returns:

/ *Type*: dict /

Configuration dictionary compatible with EventBusClient.

35.2 Class: ProcessHubMessage

Imported by:

```
from ProcessHub.transport.eventbus_transport import ProcessHubMessage
```

Message wrapper for ProcessHub data.

Wraps a dictionary payload for transport via EventBusClient.

35.2.1 Method: from_data

Create message from deserialized data.

Arguments:

- data
/ *Condition*: required / *Type*: Any /
Deserialized data from transport.

Returns:

/ *Type*: ProcessHubMessage /
ProcessHubMessage instance wrapping the data.

35.2.2 Method: get_value

Get message payload.

Returns:

/ *Type*: dict /
Dictionary payload.

35.3 Class: EventBusTransport

Imported by:

```
from ProcessHub.transport.eventbus-transport import EventBusTransport
```

EventBus (RabbitMQ) transport implementation.

Uses EventBusClient for message passing with topic-based routing.

Usage:

```
# With config file
transport = EventBusTransport(config_path="config.jsonp")
transport.register_handler("MY_TOPIC", my_handler)
transport.start()
transport.send("MY_TOPIC", {"key": "value"})
transport.stop()

# With inline config
transport = EventBusTransport(
    host="localhost",
    port=5672,
    exchange_name="process_hub",
)

# With EventBusConfig
config = EventBusConfig(host="rabbitmq.example.com")
transport = EventBusTransport(config=config)
```

35.3.1 Method: register_handler

Register a message handler for a topic.

Arguments:

- topic
/ *Condition*: required / *Type*: str | Enum /
Topic name (string or Enum).

- `handler`
/ *Condition*: required / *Type*: Callable[[Any], None] /
Callback function that receives message data (dict).

35.3.2 Method: start

Start the transport.

Connects to RabbitMQ and subscribes to registered topics.

35.3.3 Method: stop

Stop the transport.

Disconnects from RabbitMQ and cleans up resources.

35.3.4 Method: send

Send a message (blocking).

Arguments:

- `topic`
/ *Condition*: required / *Type*: str | Enum /
Topic name.
- `message`
/ *Condition*: required / *Type*: Any /
Message data (dict or dataclass).
- `target`
/ *Condition*: optional / *Type*: str / *Default*: None /
Optional target panel ID (included in message).

35.3.5 Method: send_async

Send a message (non-blocking).

Arguments:

- `topic`
/ *Condition*: required / *Type*: str | Enum /
Topic name.
- `message`
/ *Condition*: required / *Type*: Any /
Message data (dict or dataclass).
- `target`
/ *Condition*: optional / *Type*: str / *Default*: None /
Optional target panel ID (included in message).

35.3.6 Method: is_started

Check if transport is started.

Returns:

/ *Type*: bool /

True if transport is started, False otherwise.

35.3.7 Method: `is_connected`

Check if connected to RabbitMQ.

Returns:

/ Type: bool /

True if connected, False otherwise.

Chapter 36

inmemory_transport.py

In-memory transport for testing.

Provides a simple transport that delivers messages synchronously within the same process. Useful for unit testing without ZMQ.

36.1 Class: InMemoryTransport

Imported by:

```
from ProcessHub.transport.inmemory_transport import InMemoryTransport
```

In-memory transport for unit testing.

Messages are delivered synchronously within same process.

Provides test helpers to inspect sent messages.

Usage:

```
transport = InMemoryTransport()
transport.register_handler("TOPIC", my_handler)
transport.start()
transport.send("TOPIC", {"data": "value"})
# my_handler is called synchronously
# Test helper
messages = transport.get_sent_messages("TOPIC")
assert len(messages) == 1
```

36.1.1 Method: start

Start the transport (no-op for in-memory).

36.1.2 Method: stop

Stop the transport (no-op for in-memory).

36.1.3 Method: register_handler

Register message handler for topic.

36.1.4 Method: send

Send message synchronously

Stores message and invokes all registered handlers.

36.1.5 Method: `send_async`

Send message (same as `send` for in-memory).

36.1.6 Method: `get_sent_messages`

Get sent messages for testing.

Arguments:

- `topic`
/ *Condition*: optional / *Type*: `str` / *Default*: `None` /
Filter by topic (`None` for all).

Returns:

/ *Type*: `list[tuple[str, Any, str | None]]` /
List of (topic, message, target) tuples.

36.1.7 Method: `clear`

Clear all sent messages and handlers.

36.1.8 Method: `clear_messages`

Clear sent messages only (keep handlers).

36.1.9 Method: `message_count`

Get count of sent messages.

Chapter 37

zmq_transport.py

ZMQ transport implementation.

This module provides a ZeroMQ-based transport for the process hub, wrapping the ZMQ PUB/SUB pattern for message passing.

Requires: pyzmq

```
pip install pyzmq
```

Usage:

```
from ProcessHub.transport.zmq_transport import ZmqTransport
# Server (starts broker)
transport = ZmqTransport(start_broker=True)
transport.register_handler("TOPIC", my_handler)
transport.start()
# Client (connects to existing broker)
transport = ZmqTransport(start_broker=False)
transport.start()
transport.send("TOPIC", {"data": "value"})
```

37.1 Class: TcpKeepaliveConfig

Imported by:

```
from ProcessHub.transport.zmq_transport import TcpKeepaliveConfig
```

TCP keepalive configuration for ZMQ sockets.

TCP keepalive helps detect dead connections when the server crashes without sending a proper shutdown notification.

Usage:

```
# Default: detect dead connection in ~25 seconds config = TcpKeepaliveConfig()
# Faster detection: ~13 seconds config = TcpKeepaliveConfig(idle=5, interval=2, count=4)
# Disable keepalive config = TcpKeepaliveConfig(enabled=False)
```

Attributes:

```
enabled: Enable TCP keepalive (default: True)
idle: Time in seconds before sending first probe (default: 10)
interval: Time in seconds between probes (default: 5)
count: Number of failed probes before connection is dead (default: 3)
```

Detection time = idle + (interval * count) Default: 10 + (5 * 3) = 25 seconds

37.1.1 Method: detection_time

Calculate the maximum time to detect a dead connection.

Returns:

/ Type: float /

Detection time in seconds.

37.2 Class: ZmqBroker

Imported by:

```
from ProcessHub.transport.zmq.transport import ZmqBroker
```

ZeroMQ XPUB/XSUB broker (proxy).

Routes messages between publishers and subscribers.

Runs in a background thread.

Usage:

```
broker = ZmqBroker()
broker.start()
# ... use transport ...
broker.stop()
```

37.2.1 Method: xpub_endpoint

Get the XPUB endpoint for subscribers.

37.2.2 Method: xsub_endpoint

Get the XSUB endpoint for publishers.

37.2.3 Method: start

Start the broker in a background thread.

37.2.4 Method: stop

Stop the broker.

37.3 Class: ZmqAsyncClient

Imported by:

```
from ProcessHub.transport.zmq.transport import ZmqAsyncClient
```

Async ZeroMQ PUB/SUB client.

Runs an asyncio event loop in a background thread for non-blocking I/O.

37.3.1 Method: register_topic

Register a callback for a topic.

Args:

topic: Topic string to subscribe to
callback: Function to call when message received

37.3.2 Method: start

Start the client in a background thread.

37.3.3 Method: send_message

Send a message (blocking).

Args:

topic: Topic to publish to
message: Message data
timeout: Timeout in seconds

Returns:

True if sent successfully

37.3.4 Method: send_message_async

Send a message (non-blocking).

Args: topic: Topic to publish to message: Message data

37.3.5 Method: stop

Stop the client.

37.4 Class: ZmqTransport

Imported by:

```
from ProcessHub.transport.zmq-transport import ZmqTransport
```

ZeroMQ transport implementation.

Wraps ZmqBroker and ZmqAsyncClient to implement the TransportBase interface.

Usage:

```
# Server side (with broker)
transport = ZmqTransport(start_broker=True)
transport.register_handler("MY_TOPIC", my_handler)
transport.start()
transport.send("MY_TOPIC", {"key": "value"})
transport.stop()

# Client side (connect to existing broker)
transport = ZmqTransport(start_broker=False)
```

```
transport.register_handler("MY_TOPIC", my_handler)
transport.start()
# With custom keepalive (faster detection)

transport = ZmqTransport( start_broker=False, keepalive=TcpKeepaliveConfig(idle=5, interval=2,
    count=4),
    )
```

37.4.1 Method: register_handler

Register a message handler for a topic.

37.4.2 Method: start

Start the transport.

37.4.3 Method: stop

Stop the transport.

37.4.4 Method: send

Send a message (blocking).

37.4.5 Method: send_async

Send a message (non-blocking).

37.4.6 Method: is_started

Check if transport is started.

Chapter 38

`__init__.py`

UI layer - decoupled views for hub state.

Chapter 39

base.py

Abstract view interface.

This module implements improvement 3.6:

- Define abstract interface for hub state observers
- Allow pluggable views (console, web, null)

39.1 Class: HubView

Imported by:

```
from ProcessHub.ui.base import HubView
```

Protocol for hub state observers.

Improvement 3.6: Expose `get_state()` on core, have independent loop poll/observe.

This protocol allows different view implementations:

- ConsoleView: ASCII table in terminal
- NullView: No-op for headless/embedded use
- WebView: Future web-based UI
- MetricsView: Push to monitoring systems

39.1.1 Method: `on_state_changed`

Called when hub state changes.

This is for push-based updates. Views can use this to trigger re-renders or update internal state.

39.1.2 Method: `render`

Render current state.

This is for pull-based updates. The server loop calls this periodically with the latest snapshot.

39.2 Class: HubViewBase

Imported by:

```
from ProcessHub.ui.base import HubViewBase
```

Base class for hub views.

Provides default implementation where `on_state_changed` simply calls `render`.

39.2.1 Method: `render`

Render current state to output.

39.2.2 Method: `on_state_changed`

Default: just render.

Chapter 40

console_view.py

Console TUI view - decoupled from server.

This module implements improvement 3.6:

- Moved out of ProcessHubServer.run()
- Takes HubStateSnapshot instead of raw dicts
- Can be replaced with rich/textual for better TUI

40.1 Class: ConsoleView

Imported by:

```
from ProcessHub.ui.console-view import ConsoleView
```

Console-based TUI for process hub.

Displays connections and process state in ASCII table format.

Updates are non-blocking (just print to stdout).

Example output:

```
=== Process HUB ===
```

```
Process Hub is listening for process updates...
```

```
Panel ID      Session ID
-----
```

```
panel_1      session_abc
panel_2      session_abc
```

```
PID          OWNER                STATUS      PROCESS
-----
1234         panel_1                 running    system_manager
5678         panel_1,panel_2         running    edge_compute
```

```
Press Ctrl+C to exit.
```

40.1.1 Method: clear

Clear console screen.

40.1.2 Method: render

Render hub state to console.

Chapter 41

null_view.py

Null view for headless operation.

Use this when running process hub without a UI, such as in embedded mode or background service.

41.1 Class: NullView

Imported by:

```
from ProcessHub.ui.null_view import NullView
```

No-op view for headless/embedded use.

Does nothing on render - useful for:

- Running as background service
- Embedding in other applications
- Unit testing without console output

41.1.1 Method: render

No-op render.

Chapter 42

web_view.py

Web-based view for Process Hub dashboard.

This module provides a web UI for monitoring the Process Hub:

- Real-time process status display
- Connection monitoring
- Restart state visualization
- Auto-generated API documentation at /docs

Requires: fastapi, uvicorn

```
pip install fastapi uvicorn
```

Usage:

```
from ProcessHub.ui.web_view import WebView
view = WebView(host="0.0.0.0", port=2507)
server = ProcessHubServer(transport=transport, view=view)
server.run()
# Open browser to http://localhost:2507
# API docs at http://localhost:2507/docs
```

42.1 Class: `WebViewLogHandler`

Imported by:

```
from ProcessHub.ui.web_view import WebViewLogHandler
```

Custom logging handler that captures logs to a deque buffer.

Used by `WebView` to display logs in the console view.

42.1.1 Method: `emit`

Emit a log record to the buffer.

42.2 Class: `WebView`

Imported by:

```
from ProcessHub.ui.web_view import WebView
```

Web-based view for Process Hub using FastAPI.

Provides a browser-accessible dashboard for monitoring processes.

Includes auto-generated API documentation at /docs.

Usage:

```
view = WebView(host="0.0.0.0", port=2507)
server = ProcessHubServer(transport=transport, view=view)
server.run()
# Dashboard: http://localhost:2507
# API docs: http://localhost:2507/docs
```

42.2.1 Method: start

Start the web server in a background thread.

42.2.2 Method: stop

Stop the web server.

42.2.3 Method: render

Update the current state (called by server's main loop).

Arguments:

- snapshot
/ *Condition*: required / *Type*: HubStateSnapshot /
Current hub state snapshot.

42.2.4 Method: url

Get the dashboard URL.

42.2.5 Method: docs_url

Get the API documentation URL.

Chapter 43

Glossary

43.1 Glossary

Broker A ZMQ component that routes messages between publishers and subscribers using XPUB/XSUB sockets.

Client A ProcessHubClient instance that connects to a ProcessHubServer to request process operations.

Core The ProcessHubCore component containing pure business logic without I/O dependencies.

Executor A component responsible for starting and stopping processes on the operating system.

Panel A unique identifier for a client connection, used to track which clients need which processes.

Process An operating system process managed by ProcessHub.

Registry The ProcessRegistry component that maintains the state of all managed processes.

Requester A panel that has requested a specific process to be running.

Session A grouping of related panels that share process dependencies.

Snapshot An immutable copy of the hub state used for thread-safe UI rendering.

Topic A message category used for ZMQ PUB/SUB message routing.

Transport An abstraction layer for message passing between server and clients.

View A UI component that renders the hub state (ConsoleView, NullView).

ZMQ ZeroMQ, a high-performance asynchronous messaging library.

Chapter 44

Appendix

44.1 Appendix

44.1.1 License

ProcessHub is licensed under the Apache License, Version 2.0.

You may obtain a copy of the License at: <http://www.apache.org/licenses/LICENSE-2.0>

44.1.2 References

- ZeroMQ: <https://zeromq.org>
- psutil: <https://psutil.readthedocs.io>
- Python: <https://www.python.org>

44.1.3 Repository

Source code: <https://github.com/test-fullautomation/python-process-hub>

Issue tracker: <https://github.com/test-fullautomation/python-process-hub/issues>

44.1.4 Contact

Author: Nguyen Huynh Tri Cuong

Email: Cuong.NguyenHuynhTri@vn.bosch.com

Chapter 45

History

45.1 History

45.1.1 Version 1.1.0 - 06.02.2026

Fleet Orchestrator – multi-hub coordination overlay for managing multiple ProcessHub instances across PCs from a central point.

New Features

- **Fleet Orchestrator:** Central coordinator for hub discovery, health monitoring, and command routing across distributed ProcessHub instances
- **Hub Agent:** Lightweight sidecar alongside each ProcessHubServer that sends heartbeats, status reports, and handles fleet commands via the server’s public API
- **Hub Registry:** Thread-safe registry with health state transitions (online → degraded → offline) based on heartbeat timeouts
- **Fleet Client:** High-level API for CI/CD pipelines and CLI tools, supporting single-hub commands and batch operations (start/stop/reset across all hubs)
- **Fleet Web API:** Optional REST API and HTML dashboard for fleet monitoring (FastAPI-based, served at configurable host/port)
- **Fleet Protocol Messages:** Typed dataclass messages extending MessageBase – HubAnnounce, HubRegistered, HubDeregistered, HubStatusReport, FleetCommand, FleetCommandResult, FleetStatusRequest, FleetStatusResponse
- **Fleet State Snapshots:** Frozen dataclasses (HubSnapshot, FleetStateSnapshot) with tuples for thread-safe concurrent access
- **Fleet Topics:** FleetTopics enum with FLEET_ prefix for EventBus routing key separation from local hub topics
- **Event Callbacks:** FleetClient supports on_hub_online and on_hub_offline callbacks for reactive fleet monitoring

Architecture

- Fleet is a pure **overlay** – no modifications to core/, runtime/, or transport/ modules
- Each local ProcessHub works independently; the orchestrator is optional
- HubAgent uses only the server’s public API (get_state_snapshot, handle_start_request, handle_stop_request, reset)
- Communication uses EventBus (RabbitMQ) with fleet.* routing key prefix in production, InMemoryTransport for testing

Documentation

- Fleet Orchestrator plan document (`docs/FLEET_ORCHESTRATOR.md`)
- Architecture Decision Records: ADR-017 (Overlay Pattern), ADR-018 (Heartbeat Health), ADR-019 (Command Routing), ADR-020 (Frozen Snapshots)
- PlantUML diagrams: `fleet_overview`, `fleet_component`, `class_fleet`, `sequence_fleet_discovery`, `sequence_fleet_command`, `state_hub_health`
- Examples: `12_fleet_basic`, `13_fleet_ci_integration`, `14_fleet_web_dashboard`

Installation

- Added `fleet` optional dependency group: `pip install python-process-hub[fleet]`
- Fleet web API requires FastAPI and uvicorn
- Optional imports with `HAS_FLEET` flag in `ProcessHub.__init__`

Tests

- 76 new fleet tests across 5 test files (`models`, `hub_registry`, `hub_agent`, `orchestrator`, `fleet_client`)
- All tests use `InMemoryTransport` – no external dependencies required
- Total test count: 183 tests (107 existing + 76 fleet)

45.1.2 Version 1.0.0 - 15.01.2026

Initial release of `ProcessHub` as an independent package.

- Migrated from `ta-framework-tsb` `process_hub` module
- Clean architecture with separated core, transport, and runtime layers
- ZMQ transport with XPUB/XSUB broker
- In-memory transport for testing
- Console view for TUI display
- Process execution with platform-specific implementations
- Restart coordination state machine
- Logging server integration (Fluentbit, Telegraf)
- Comprehensive test suite (107 tests)
- Extended setup with `CRepositoryConfig`

ProcessHub.pdf

Created at 07.02.2026 - 20:22:54

by GenPackageDoc v. 0.16.0
